

Untangling the Timeline: Challenges and Opportunities in Supporting Version Control in Modern Computer-Aided Design

Yuanzhe Deng
Mechanical and Industrial
Engineering
University of Toronto
Toronto, Ontario, Canada
yuanzhe.deng@mail.utoronto.ca

Shutong Zhang
Electrical and Computer Engineering
University of Toronto
Toronto, Ontario, Canada
Computer Science
Stanford University
Palo Alto, California, USA
tonyzst@stanford.edu

Kathy Cheng
Mechanical and Industrial
Engineering
University of Toronto
Toronto, Ontario, Canada
kathy.cheng@mail.utoronto.ca

Alison Olechowski
Mechanical and Industrial
Engineering
University of Toronto
Toronto, Ontario, Canada
a.olechowski@utoronto.ca

Shurui Zhou
ECE&CS
University of Toronto
Toronto, Ontario, Canada
shurui.zhou@utoronto.ca

Abstract

Version control is critical in mechanical computer-aided design (CAD) to enable traceability, manage product variation, and support collaboration. Yet, its implementation in modern CAD software as an essential information infrastructure for product development remains plagued by issues due to the complexity and interdependence of design data. This paper presents a systematic review of user-reported challenges with version control in modern CAD tools. Analyzing 170 online forum threads, we identify recurring socio-technical issues that span the management, continuity, scope, and distribution of versions. Our findings inform a broader reflection on how version control should be designed and improved for CAD and motivate opportunities for tools and mechanisms that better support articulation work, facilitate cross-boundary collaboration, and operate with infrastructural reflexivity. This study offers actionable insights for CAD software providers and highlights opportunities for researchers to rethink version control.

CCS Concepts

• **Applied computing** → **Computer-aided design**; • **Social and professional topics** → **Project management techniques**; • **Human-centered computing** → **Empirical studies in collaborative and social computing**.

Keywords

Product Data Management, Version Control, Information Infrastructure

ACM Reference Format:

Yuanzhe Deng, Shutong Zhang, Kathy Cheng, Alison Olechowski, and Shurui Zhou. 2026. Untangling the Timeline: Challenges and Opportunities in Supporting Version Control in Modern Computer-Aided Design. In *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems (CHI '26)*, April 13–17, 2026, Barcelona, Spain. ACM, New York, NY, USA, 20 pages. <https://doi.org/10.1145/3772318.3791061>

1 Introduction

With the growing complexity of engineering design, modern hardware product development increasingly depends on computer-aided design (CAD) software. With mechanical CAD, designers model the 3D geometric details of products with parametric commands and constraints, assemble parts for analysis, iterate designs, and deliver finalized specifications to downstream manufacturers [108]. While CAD originated as a digital substitute for pencil-and-paper drafting, it has since evolved into a virtual design studio where geographically distributed engineers discuss, argue, negotiate, and form consensus around design decisions [83, 98]. As CAD projects scale, they exhibit the hallmarks of a complex system as Boy defines [11]: intricate interdependencies among components [25], increasingly distributed and collaborative workflows [23], sophisticated data management tooling [115], and the nonlinear design processes resulting from parallel development, which must be reconciled [22]. CAD has thus become not merely a technical drafting tool built for individual engineers but a complex socio-technical system at the centre of modern collaborative engineering and making.

Throughout the product development process, design iterations are fundamental to exploring the design space and resolving failures, and the frequency of this iteration is strongly tied to design performance [119, 122]. Yet, these iterations generate rapidly expanding version histories that demand effective data management, capable of handling the complex interdependent CAD artifacts [42, 61]. At the same time, collaboration across organizational and system boundaries further amplifies the system complexity [19], especially



This work is licensed under a Creative Commons Attribution 4.0 International License. *CHI '26, Barcelona, Spain*

© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2278-3/26/04
<https://doi.org/10.1145/3772318.3791061>

as each CAD system operates with different proprietary file formats [57, 61]. Consequently, version control has become an essential information infrastructure in CAD that structures relations among design versions, coordinates distributed work, and provides the context for effective communication [23, 49, 105, 106]. Like any infrastructure, it remains largely invisible to the users until breakdown [106], where such breakdowns may lead to consequences such as downstream manufacturing errors due to poor traceability [56], loss of functionality due to broken component dependencies [2], and chaotic coordination triggered by mismanaged branches of parallel development [22]. These breakdowns hence motivate a systematic examination of the socio-technical challenges of modern CAD collaboration from the perspective of end users, with a particular focus on version control as an information infrastructure.

While project failures and collaboration inefficiencies rarely stem from purely technical faults, they often arise from misalignment between technology and the social and organizational complexity of the environment in which it is deployed [8]. For example, version control in CAD is traditionally implemented to be centralized and file-based, which requires users to download files, make changes locally, and then re-upload new versions to a central server. While this structure effectively minimizes editing conflicts, it restricts parallel work and slows distributed collaboration [23]. However, as new CAD platforms such as *ONSHAPE* enable real-time co-editing to mitigate these bottlenecks [5], studies suggest that synchronous collaboration may increase communication overhead and coordination burdens [4, 90]. The optimal implementation is non-obvious, and ongoing studies are necessary. Despite growing research on version control in CAD (e.g., traceability [23, 118], branching and merging [22, 113], synchronous collaboration [4, 21, 90]), there lacks a systematic understanding of the user challenges in version control, especially across the diverse landscape of modern CAD platforms [60]. Without such an understanding, it is difficult to derive design principles for next-generation CAD infrastructures.

Despite known technical limitations, users routinely restructure their workflows around existing technology, and infrastructures cannot be understood or improved without attending to these creative and improvisational practices of ordinary users [95]. For instance, consider the hobbyist open-source hardware (OSH) community, where contributors frequently share 3D printing-ready CAD models in diverse proprietary formats on platforms such as *THINGIVERSE*, which provide little to no version control support [26, 76]. To compensate for this infrastructural limitation in tracking design versions, OSH hobbyists repurpose the model description field to document change history, despite the space being intended for functional summaries and 3D printing instructions [26]. Bottom-up workarounds like this highlight the gap between formal infrastructure and actual practice; this study surfaces and synthesizes the real-world challenges users encounter as well as their creative workarounds when performing version control in CAD.

In this paper, we address several important socio-technical questions to establish a foundation from which to innovate: Where does current CAD infrastructure fall short in supporting collaborative design; how do users navigate fragmented versioning workflows; and how might future CAD systems better support traceability, design variation, and distributed coordination? To answer these questions, we conducted a qualitative analysis of 170 version-control-related

user posts and their associated discussions, collected from seven CAD-related online forums. Through inductive content analysis, we identified four high-level categories of version control challenges, concerning the management, continuity, scope, and distribution of versions. Building on these findings, we derive three high-level design opportunities for advancing version control support in CAD, including better supporting articulation work throughout the design process, facilitating cross-boundary collaboration, and developing version control that foregrounds reflexivity as a key design principle. Together, these insights re-emphasize CAD not merely as a technical tool but as a socio-technical system whose technical development must be shaped by the practical realities of its use.

In summary, we contribute the following insights to the growing CAD community within the field of human-computer interaction (HCI):

- An empirical investigation of the user challenges in performing version control with CAD, based on public discussions among users across seven online forums.
- A comparison of functionalities across three commercially available CAD software platforms in support of the identified version control challenges.
- A set of design opportunities for version control in CAD as an information infrastructure.

2 Background and Related Work

Originating in software engineering [28, 99], the concept of version control has since expanded to domains such as collaborative creative practices [110], scientific workflow management [1], and modern engineering design in CAD. In this section, we review the types of version control system (VCS) used in CAD (Section 2.1) and their applications (Section 2.2), followed by the research gap in Section 2.4.

2.1 Types of Version Control

The complexities of CAD as a design platform result in particularly vexing challenges [2, 28, 117], and have resulted in varied implementations of version control.

2.1.1 Centralized version control. Traditional version control in CAD is largely supported through a *local centralized version control system* (CVCS), often via product data management (PDM) or product lifecycle management (PLM) systems [28, 72, 107], which originated from software development tools like CVS¹ and SUBVERSION². In this model, a central server maintains the complete version history, and users acquire access to the versions as a client through network connection [99]. To modify a CAD file, users must first retrieve the file from the server (i.e., “checking out” the model) to their local machine with the CAD software installed, serving to lock the file and preventing concurrent edits. When complete, the user “checks back in” the updated version to the server.

As cloud infrastructure matures, PDM systems have also transitioned to *cloud-based CVCS* [54], enabling distributed access to CAD model data and eliminating the overhead cost of infrastructure setup [59]. This shift facilitates remote collaboration across

¹<https://cvs.nongnu.org>

²<https://subversion.apache.org>

geographic locations [6]. However, the fundamental version control workflow remains centralized, where copies of CAD files must be checked out, modified locally, and uploaded in full after every version update.

Although the check-in/check-out workflow in CVCS avoids editing conflicts, it introduces delays and limits concurrent editing. Users often only become aware of design changes after a file is checked in, sometimes leading to integration issues when interdependent components are developed in parallel [10, 23]. Despite these limitations, CVCS remains the dominant approach in modern CAD, especially when exchanging models across different CAD platforms or in open-source environments [26]. While prior research has documented some inefficiencies in CVCS workflows, a systematic understanding of how these technical constraints impact user collaboration is lacking. In this paper, we review the challenges from the users' perspectives and examine how they adapt to these limitations in practice.

2.1.2 Distributed version control. In contrast, distributed version control systems (DVCS) like Grr³ allow users to work independently by maintaining isolated local repositories, supporting parallel development and more flexible workflows. DVCS tracks individual changes at a finer granularity (e.g., line-level differences), resulting in smaller and more cohesive commits with improved traceability [13]. Modern CAD software achieves *cloud-based DVCS* by migrating both the CAD software and data to the cloud [42]. This architecture enables users to access modelling tools directly through a web browser, where every design action is continuously synchronized with the cloud. Rather than uploading whole files, changes are incrementally recorded and integrated in real-time, eliminating manual check-in/check-out [64, 87]. These systems support simultaneous multi-user collaboration on the same model, enabling a new class of cloud CAD or multi-user CAD workflows [5].

Despite these advances, the implications of DVCS in CAD remain under-explored [33, 43]. While DVCS resolves key limitations of CVCS (i.e., low concurrency in collaboration), it introduces new challenges. For instance, Cheng et al. observed that synchronous editing can result in design conflicts and duplicate work due to insufficient awareness of teammates' actions [24]. Besides, most existing studies on cloud CAD focus on examining design quality through surface-level metrics such as user satisfaction, communication patterns, and telemetric action data [34, 40, 90, 91, 127], rather than analyzing how version control mechanisms influence collaborative workflows. In this paper, we systematically examine the user-facing challenges across both centralized and distributed VCS, and we discuss the implications for future CAD software providers.

2.2 Use of Version Control

In order to review the uses of version control in CAD, we apply Estublier and Casallas' three orthogonal classes of versioning, established in the context of software engineering: historical, logical, and cooperative, each with different motivations and circumstances [39].

2.2.1 Historical versioning for traceability. Historical versioning records the linear progression of a project in the time dimension,

enabling traceability and accountability [39, 120]. In CAD, VCS captures design milestones as immutable snapshots, preserving the evolution of a model throughout its lifecycle [62]. These versions provide essential backup points when fixing modelling errors and facilitate design reuse when revisited. Ideally, these snapshots are semantically meaningful representations of the design history [28, 55].

Maintaining high-quality traceability is resource-intensive in both software [65, 80] and CAD [23], particularly in large-scale projects with complex interdependencies. In CAD, traceability is further complicated by the binary and proprietary nature of the model files, which hinders efficient comparison between versions [23]. OSH communities, where contributors frequently use different tools and platforms, face additional challenges due to increased communication overhead and inconsistent version tracking practices [26]. Although the importance of traceability is well-established in case studies [26] and technology review articles [42, 61, 94], the specific burdens and breakdowns encountered by general CAD users remain under-analyzed. This motivates a closer investigation into how different types of VCSs (as reviewed in Section 2.1) support or hinder traceability in real-world CAD workflows.

2.2.2 Logical versioning for variation. Logical versioning supports the coexistence of multiple variants of a design, each representing a parallel evolution path of the project [39]. In software engineering, this involves *branching* (creating copies of) the source file for isolated development and *merging* (integrating) changes back into a target branch [7, 96]. In CAD, branching is used mainly to manage product lines (e.g., product releases, design variants), isolate development risk, and coordinate work among collaborators [22].

In CAD, managing variants becomes increasingly complex as the number of branches grows [22], as is the case in software [71, 92]. Branching in CAD introduces unique challenges due to the tightly coupled geometric and parametric dependencies among models. Unlike in text-based source code, where developers can selectively merge specific changes (i.e., cherry-picking⁴), branching in CAD typically results in entire model variants being duplicated, without fine-grained control over which features or modifications are preserved, discarded, or merged [22]. Although recent advancements in cloud CAD have made branching and merging more accessible, this functionality also introduces new coordination demands. In practice, an "integrator" role often emerges, responsible for resolving merging conflicts and maintaining up-to-date model versions across branches [4]. Extending beyond Cheng et al.'s review [22] on the use cases and deficiencies of branching in CAD, this paper systematically reviews how both branching and merging are supported and hindered in practice.

2.2.3 Cooperative versioning for synchronization. Cooperative versioning supports real-time collaboration by synchronizing changes across users [82]. Cloud-based CAD systems automatically create implicit versions when changes are detected, enabling synchronous updates across devices and locations. These automatically created implicit versions complement the explicit versions used for historical and logical purposes, offering a layered versioning model that balances flexibility and control. As discussed in Section 2.1.2,

³<https://git-scm.com>

⁴<https://git-scm.com/docs/git-cherry-pick>

cloud-based DVCS eliminates the check-in/check-out bottleneck of traditional CVCS and enables real-time multi-user editing. However, this increased synchronicity introduces new challenges, and a comprehensive review is lacking in understanding the collaborative trade-offs introduced by these advances in VCS.

2.3 Fundamental Differences Between CAD and Software Version Control

Although version control originates in software engineering, the challenges of managing, interpreting, and coordinating versions in CAD cannot be captured by existing software-centric VCS models. From an HCI perspective, CAD introduces fundamentally different forms of representational friction and socio-technical limitations that reshape how people collaborate and make sense of evolving artifacts:

- *Geometric complexity.* CAD models are built through sequences of parametric operations and constraint networks that implicitly encode design intent: a single dimension change may cascade across multiple parts and assemblies [18, 25, 94]. This makes various sophisticated operations that are fundamental in software VCS (e.g., diff, merge) not merely unsupported in traditional CAD systems but conceptually misaligned with how CAD artifacts behave in their current forms [15, 42, 81].
- *Versioning workflows.* While traditional engineering change management practices document detailed justifications of each revision [56, 121], modern cloud-based CAD systems streamline the process and generate implicit versions continuously to enable synchronous collaboration, creating dense but low-semantic version histories [5]. Unlike text-based code versions that can be queried, compared, and navigated, CAD versions require users to reconstruct the design intent through geometric inspection and temporal inference. Without this labour, the version history quickly loses its meaning and becomes impossible to navigate [23].
- *Collaboration span.* CAD assemblies rely on references to components that span multiple files and off-the-shelf parts imported from other platforms [25, 29]. However, when CAD models are transferred across projects or exported to lightweight neutral formats, parametric information and design history are often stripped entirely to reduce the file size, leaving only “dumb” geometries that are not editable [61]. These breakdowns do not manifest as textual conflicts in source code, and therefore the ways users negotiate meaning, ownership, and repair in CAD extend beyond what software VCS research typically examines.

Taken together, these representational complexities and collaborative demands show that CAD VCS cannot be understood as a simple instance of software VCS in another domain. Instead, it constitutes a distinct class of HCI challenges involving cross-representation sensemaking, coordination around interdependent artifacts, and the management of complex geometry-based design histories. These differences motivate the need for a user-centred investigation of how people experience and work around CAD-specific version control breakdowns, and how future systems might better support CAD and other complex design workflows.

2.4 Research Gap

Despite the central role of version control in collaborative design, existing research on CAD versioning remains fragmented. Prior work has largely focused on technical implementations of VCS or has studied collaboration in CAD without in-depth analysis of the supports and limitations of the VCS. There remains a lack of systematic cross-platform understanding of the socio-technical challenges users face when applying version control in real-world collaborative practices in CAD. As the product design community acknowledges the importance of version control during product development, and an increasing number of businesses have integrated the use of version control in their daily design activities, a review of user feedback is required. This study addresses that gap by synthesizing user-reported experiences and empirical findings to examine where current CAD version control workflows succeed, where they fall short, and what users need to support more effective and collaborative design.

3 Methodology

In this section, we present an overview of our study scope and descriptions of our data collection and analysis methods.

3.1 Study Scope

In order to effectively understand the varying use cases and challenges in version control for mechanical CAD, we primarily focused on studying the users’ online discussions from three model creation platforms (Section 3.1.1), two model sharing platforms (Section 3.1.2), and two general online forums that are not officially affiliated with any commercial CAD software (Section 3.1.3), as summarized in Table 1. This study expands the scope of a previous HCI study [22], which gained valuable insights on branching in CAD from a user perspective.

We chose to analyze online forum discussions because they provide candid and naturally occurring user-reported challenges of real-world CAD tool use, surfacing organically in response to breakdowns, unmet needs, and design challenges. By examining the full discussion threads rather than isolated posts, we were able to reconstruct user intent, situational constraints, and problem-solving strategies with greater context and fidelity. This approach reveals not only users’ perceptions of technical limitations, but also how users adapt, coordinate, and develop informal workarounds across a broad range of design contexts, offering ecologically valid insights into infrastructural friction, collaborative dynamics, and tool appropriation across distributed design workflows.

3.1.1 Model Creation Platforms. For product development, the official user forums of three commercially available CAD platforms were included in our study (AUTODESK FUSION, ONSHAPE, and SOLIDWORKS). These are also the only three CAD software platforms that enable built-in version control for 3D CAD models [54, 87, 103], each representing one type of VCS introduced in Section 2.1, as shown in Table 1. In other CAD software, such as OPENSCAD, CREO, and AUTODESK INVENTOR, enabling version control requires the purchase of external PDM/PLM solutions. At the same time, the selected CAD platforms not only target professional engineering

Table 1: Platforms selected for the study of version control in CAD.

Type	Platform	Launch	Intro. of VCS	VCS Type	Data Source
Model creation	SOLIDWORKS	1995	2003	Local CVCS	SOLIDWORKS Forums ^a
	AUTODESK FUSION	2013	2013	Cloud CVCS	AUTODESK Forums ^b
	ONSHAPE	2015	2015	Cloud DVCS	ONSHAPE Forums ^c
Model sharing	THINGIVERSE	2008	N/A	None	THINGIVERSE Groups ^d & REDDIT ^e
	THANGS	2020	2020	CVCS	REDDIT ^e
General forum	ENG-TIPS	1997	N/A	N/A	ENG-TIPS ^f
	CAD FORUM	2021	N/A	N/A	CAD FORUM ^g

^a <https://forum.solidworks.com/>; ^b <https://forums.autodesk.com/>; ^c <https://forum.onshape.com/>; ^d <https://www.thingiverse.com/groups/thingiverse/>;

^e <https://www.reddit.com/>; ^f <https://www.eng-tips.com/>; ^g <https://cadforum.net/> and later migrated to <https://forum.cadmunity.com/> in May 2025.

designers for commercial use, but also provide different kinds of account types for design hobbyists with reduced support of advanced features.

All three selected CAD modelling platforms host an online forum for users to post and discuss product-related topics (e.g. feature-related questions and improvement requests) as a communication platform between the users and the software providers. These forums are listed in the last column of Table 1. Access to post and participate in these forums is limited to users with a registered account with the CAD software package, which ensures the relevance and quality of the discussion.

3.1.2 Model Sharing Platforms. CAD modelling is increasingly important for hobbyists, too, with the growing popularization of 3D printing [58]. Both THINGIVERSE⁵ and THANGS⁶ attempt to lower the barrier of entry to 3D printing by providing a platform for hobbyists to quickly search for and use CAD models that suit their needs.

Most notably, THINGIVERSE has built the largest maker community for sharing 3D-printable CAD models, with over 3 million models exchanged between over 2.3 million registered users [68], making it a major platform for studies in collaborative open product development [3, 17, 26]. In THINGIVERSE, user discussions about the platform take place in different Groups, and we focused our study on the “General” forum of the “Thingiverse” group, the official Group managed by the platform.

We chose to include THANGS in this study, as it is the first to support the search for similar models across different CAD model-sharing platforms (e.g., THINGIVERSE, FREE3D), a version-control-related feature that is widely absent from other model-sharing platforms. Based on a user-uploaded model, THANGS will attempt to locate different versions of the same design [93] and provide a version control workspace to visualize the geometric difference between CAD models [116]. THANGS does not have an officially affiliated forum for discussion, but there is an official user account that is active on REDDIT,⁷ an online forum with communities organized by topic. To collect discussions around version

control on THANGS, we searched four subreddit communities where the official THANGS account actively participates: “r/3Dprinting”, “r/resinprinting”, “r/Thangs3D”, and “r/blender”.

3.1.3 Online Forums. Beyond gathering user discussions from product-specific forums, we examined two general CAD-related forums (CAD FORUM and ENG-TIPS) that are managed by third parties and are meant for users to freely share cross-platform knowledge and discussions. Here we expected posts on the generic usage of version control in CAD and comparisons of solutions between major CAD software providers. For the ENG-TIPS forum, we specifically focused on two relevant sub-forums: “Mechanical engineering general discussion” under the “Mechanical Engineers” section and “Engineering programs/apps (general) Forum” under the “Engineering Computer Programs” section.

3.2 Data Collection

Keyword search. We developed the following set of keywords to use when searching for posts and discussions in the selected online forums: “version control,” “change history,” “versioning,” “version history,” and “variation management.” These keywords were selected because they either directly refer to the use and need for version control or relate to the major applications of version control in mechanical product design settings. Initial exploratory searches using broader terms such as the term “version” alone returned more than 20,000 threads, the vast majority of which were unrelated to CAD version control but rather discussions about software compatibility and licensing issues, for example. Using a more focused set of keywords allowed us to prioritize precision and ensure that the retrieved threads specifically addressed the version-control challenges examined in this study. Nevertheless, to verify that this search strategy did not exclude major categories of discussion, we conducted iterative spot-checks using broader keyword queries (e.g., “version”, “history”). These checks did not reveal additional themes beyond those present in the final dataset, suggesting that the selected threads adequately captured the conceptual space of interest and that thematic saturation was reached.

Time window. We defined a time window of relevance for the forum threads, which began April 2005 (when version control was

⁵<https://www.thingiverse.com>

⁶<https://thangs.com>

⁷<https://www.reddit.com/user/Thangs3D/>

introduced in software development by GIT [16]) to July 2024 (when the data were collected). From the data sources presented in Table 1, the threads were collected via the workflow visualized in Figure 1. To collect REDDIT threads, we used the PROJECT ARCTIC SHIFT.⁸ For all other platforms, a self-developed web-scraper was built in Python using the BEAUTIFUL SOUP⁹ and SELENIUM¹⁰ packages to collect all the textual information. The step resulted in a total of 424 forum threads.

Web scraping. For each thread, we collected the title and content of the initial post, along with the user name of the author and the publishing date. We also collected all comments on those posts, along with the commentor’s username. This approach allowed us to infer intent, situational constraints, and user goals with greater fidelity. The total number of threads collected from each platform is tabulated in Table 2.

3.3 Data Analysis

Next, two authors of the paper manually coded the collected threads, as illustrated in Figure 1. One coder has expertise in version control for product design in CAD, and the other has expertise in version control for software development with GIT. Following the Institute of Education Science (IES) guidelines [85], a subset of 100 threads (> 20% of all threads collected) was first randomly selected for open coding [100]. The two coders began by screening the subset and excluding irrelevant threads (e.g., discussions on debugging of technical features). Based on the remaining subset, the coders developed a scheme to classify the relevant discussions into categories of user challenges, where the inter-assessor agreement (IAA) was calculated to be 83% total agreement. With the obtained IAA exceeding the minimum threshold (> 80%) proposed by the IES guidelines [85], the coders then proceeded to individually complete the analysis of the remaining dataset using the developed coding scheme, where each thread was first evaluated for relevancy and then assigned to one or more codes (i.e., user challenges), if appropriate. For the full dataset, the initial round of coding resulted in 81.4% total agreement, 9.67% partial agreement (two coders agreed on one or more codes assigned, but not all), and 8.96% disagreement. The two coders discussed and resolved all disagreements from the initial round of coding. In total, 170 threads were identified as relevant to the use of version control in CAD, where the number of threads collected per forum is reported in Table 2, and more characteristics of the dataset are visualized in Figure 2.

4 Results

Through open coding of all 170 relevant threads identified in Table 2, we derived four categories of user challenges: management, continuity, scope, and distribution of versions in CAD, the results of which are summarized in Figure 3. Challenges that were only reported in three threads or less were grouped as “Other”. These categories were derived on the basis of patterns that emerged directly from the data, rather than predefined frameworks. In this section, we present the detailed descriptions and analysis of each coding category. Where excerpts from forum threads are quoted,

Table 2: Number of threads collected and analyzed from the selected data sources.

Forums	Threads collected	Relevant threads identified
AUTODESK	135	66 (49%)
ONSHAPE	136	55 (40%)
SOLIDWORKS	67	20 (30%)
REDDIT	49	17 (35%)
CAD FORUM	28	9 (32%)
ENG-TIPS	8	3 (38%)
THINGIVERSE Groups	1	0 (0%)
Total	424	170

we reference the source of the thread with the forum name and the posting year in brackets; for example, “(OS21)” refers to a thread from the ONSHAPE forum that was posted in 2021.¹¹

4.1 Management of Version Control in CAD

This section discusses the challenges users raised in managing the accelerating number of versions created, either automatically by the software or manually by the users, over the course of a product’s development.

4.1.1 Navigating Version History. In our study, the most frequently discussed challenge concerns the navigation of a model’s version history, mentioned in 47 forum threads. CAD systems typically use an incremental numbering scheme (e.g., v1, v1.1) to trace design evolution, either assigned automatically or imposed by organizational protocols [109]. However, this naming scheme quickly becomes unmanageable, especially for CAD platforms with a CVCS. As one Autodesk user elaborates: “*In theory this works, but us humans get hammered into us to save regularly so we never lose work if there is a power failure. This means that we get way more versions than we actually need and annoying by the time it gets to V50 or more. By the time you get into these larger version numbers, the advantage of opening a specific one to go back in time becomes somewhat redundant as any one of several will do*” (AD24). This reflects a common trend across forums: version numbers increase, but they convey little useful information.

Beyond their sheer volume, numeric labels lack descriptive summaries, offering little insight into the content or purpose of each version. As a result, version timelines become difficult to navigate, particularly with limited access to detailed documentation, as illustrated in Figure 4. As one user stated, sorting through versions “*will become a nightmare and leads to this problem of having to navigate the document history to exactly where and if something was released*” (OS19), highlighting the navigational challenges for both designers and downstream manufacturers. Across all three tools shown in Figure 4, the interfaces also compress descriptions into narrow columns, obscuring contextual information unless users

⁸https://github.com/ArthurHeitmann/arctic_shift

⁹<https://www.crummy.com/software/BeautifulSoup/>

¹⁰<https://selenium-python.readthedocs.io>

¹¹Abbreviations for the seven forum sites are: AUTODESK (AD), ONSHAPE (OS), SOLIDWORKS (SW), REDDIT (RD), THINGIVERSE Groups (TV), CAD FORUM (CF), and ENG-TIPS (ET).

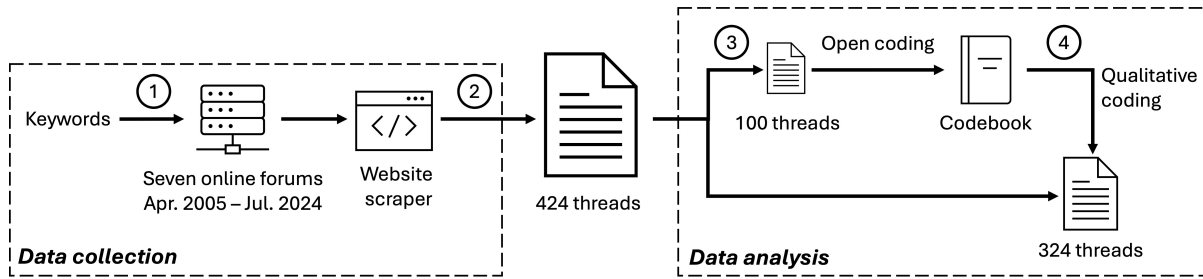


Figure 1: Overview of the research methodology. (1) A list of keywords was first defined in Section 3.2 to search for posts and discussions from seven forums. (2) A total of 424 threads from the forum search are mined with a self-developed website scraper. (3) A subset of 100 threads was first randomly selected for open coding with the method described in Section 3.3, and a coding scheme was developed. (4) The established codebook was then used to qualitatively code the remaining 324 threads.

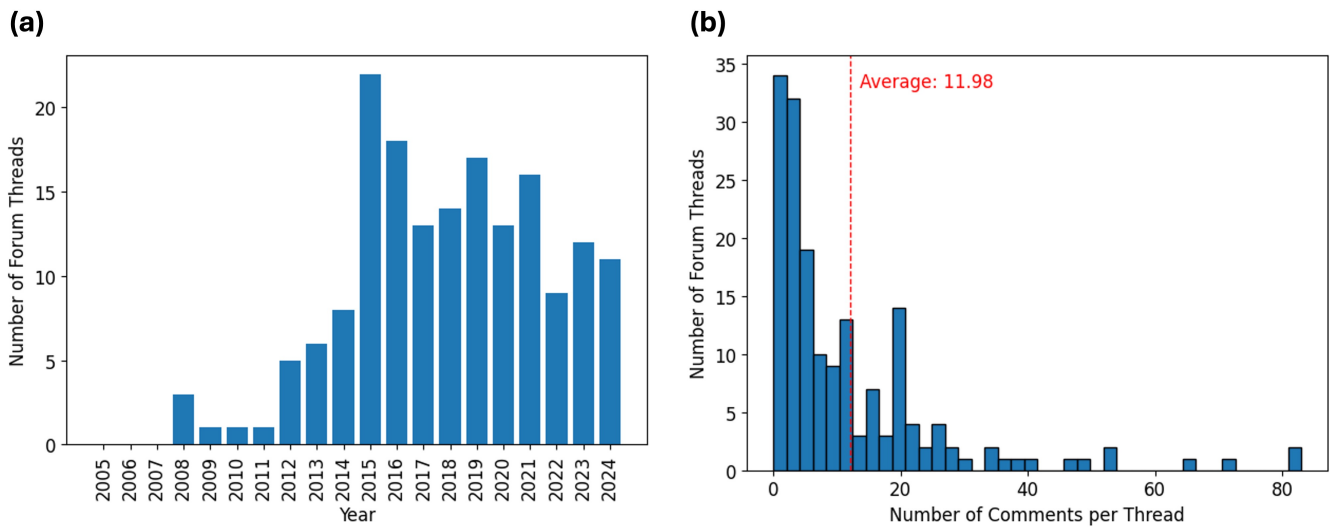


Figure 2: Characteristics of the dataset of 170 relevant forum threads included in our analysis. (a) Temporal distribution of forum threads from April 2005 to July 2024. (b) Distribution of comment counts per forum thread. On average, each thread contains 11.98 comments.

manually expand them with additional clicks. Although this interface design enables a large volume of information to be displayed within a compact view, the resulting loss of visibility slows down both design review and version retrieval processes.

Even when versions include user-written descriptions, searching for a specific design modification remains difficult. For instance, an ONSHAPE user described that “*instead of reading through 1000’s of lines and sorting through dozens of hastily made versions, I just want to find the last time I was working on a specific feature of a specific part*” (OS18). Current CAD platforms do not support semantic search across version histories; search functions are limited to rudimentary keyword matching in system-generated attributes (in ONSHAPE [52] and SOLIDWORKS [114]) or user-authored documentation (in SOLIDWORKS [114]).

4.1.2 Editing Version History. Our analysis revealed that the common cluttered version history, filled with minor or unintended

updates, not only creates navigation challenges (Section 4.1.1) but also fuels users’ desire to retroactively edit or prune the version history, altering its immutability. This issue was mentioned in 45 forum threads. Cloud-based CAD platforms exacerbate this issue by automatically generating new versions to synchronize data and prevent data loss, either periodically in AUTODESK FUSION [44] or continuously to enable DVCS in ONSHAPE [5]. While this automation ensures reliability, it also floods the version timeline with entities that do not reflect meaningful design changes. One user described this as “*the equivalent of both extreme hand holding and the software behaving like IT is in charge and not you the creator and designer*” (AD15), pointing to tensions between data integrity and user autonomy. Moreover, users often create versions unintentionally that do not include meaningful design updates, further cluttering the version history. For example, a user may “*mistakenly save a file with a section analysis visible, changing it to invisible*

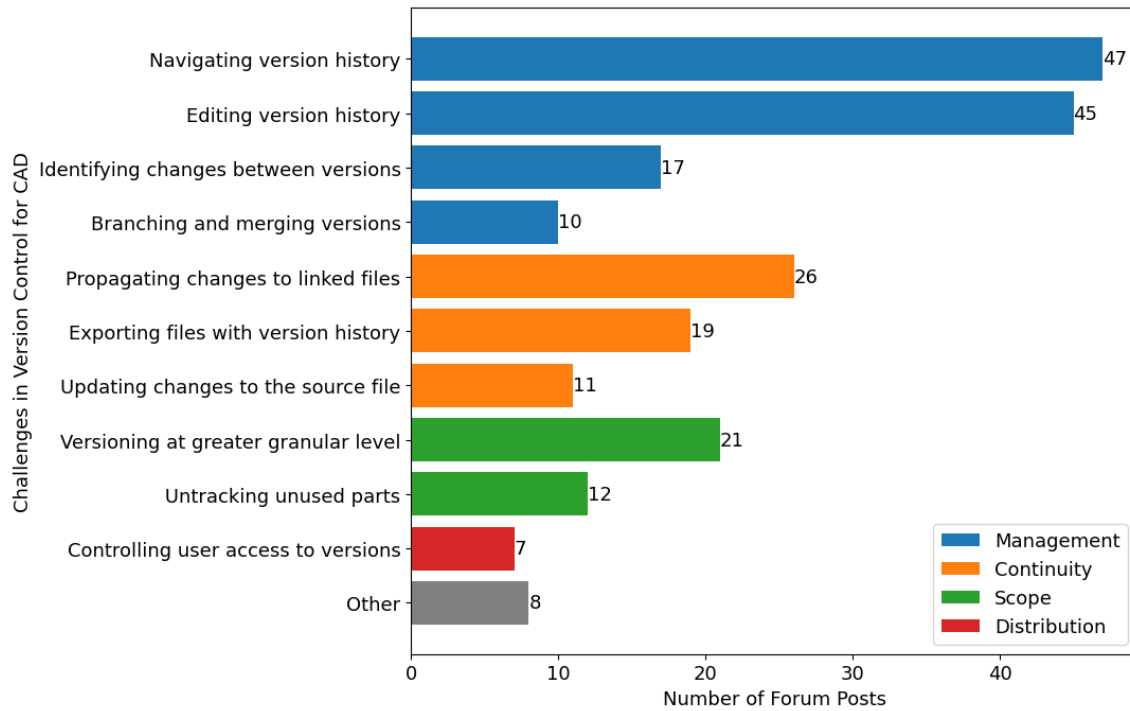


Figure 3: Frequency of challenges with using version control in CAD reported in the relevant forum threads ($n = 170$), where one or more challenges were identified in each thread. Challenges are grouped into four categories: management, continuity, scope, and distribution of version control in CAD.

acts as a major change and once saved, the version number increases” (AD18).

While version control ensures traceability and avoids editing conflicts between collaborators, the accumulation of insignificant versions complicates both review and communication of design evolution, and can also introduce regulatory and compliance risks. As a user explained: “In our regulated industry, we normally have to engage the regulators on any product change and also gain their approval on changes. This is a long process that can take weeks or months, so we cannot afford to have phantom changes propagating up through the product structure that the regulator does not understand since there is no ‘real’ change” (OS22). This highlights that the impact of noisy version histories extends beyond day-to-day navigation challenges to broader organizational and compliance processes.

Despite these frustrations, CAD platforms enforce immutability of versions once created. Editing versions in CAD carries risks not seen in textual systems like source code: changes made to an earlier point in the timeline can invalidate parametric constraints to downstream geometry, making history editing a structurally fragile operation. However, even if version deleting cannot be achieved, our analysis of forum discussions reveals users’ strong desire for “the ability to flag some versions as obsolete with an option to hide them from the version graph” (OS16). Yet, no current CAD platforms support such functionality, leaving users with bloated and static records.

4.1.3 Identifying Changes between Versions. When revisiting a CAD file’s version history, users often need to understand the semantic differences between versions, beyond geometric changes, which is a concern mentioned in 17 forum threads. As a user pointed out: “Due to the nature of CAD data (unlike software code), it is not easy to do a simple ‘compare’ to figure out changes and why. Such visual compare tools (in CAD but not so much in PDM) show ‘what’ changed. But users still have no way to know ‘why’ the changes were made, either in CAD or in PDM” (CF23). The same user further explained that “the ‘why’ and ‘what’ are really the ECOs/ECNs for the part” (CF23), where the engineering change orders (ECOs) or engineering change notices (ECNs) are part of the formal process of documenting engineering changes (the “what”) and the rationales behind making these changes (the “why”) [56, 121]. As shown in Figure 5a, b, and d, all three studied CAD model development platforms support geometry-level version comparison only. These tools list parametric edits of the modelling commands and visualize geometric differences, but offer limited support for higher-level insights such as intent or downstream implications. Interpreting these differences remains labour-intensive and error-prone, especially when changes are subtle or cumulative.

The challenge is exacerbated when files are exported and shared on platforms like THINGIVERSE and THANGS. Exported files often use proprietary or neutral exchange formats that discard the version history and the modelling operations that constructed the model (i.e., dumb geometry that loses editability). As one user noted, these

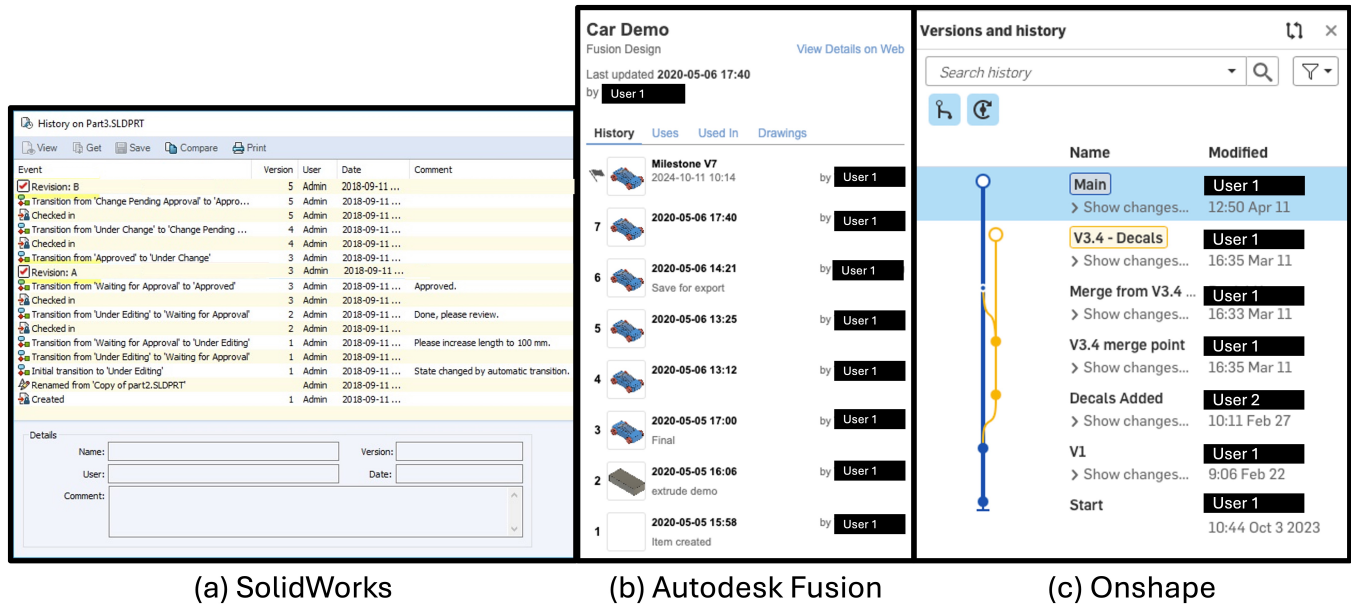


Figure 4: Display of version history of a file in the three studied CAD software. (a) SOLIDWORKS automatically generates versioning event descriptions and incremented version numbers for every PDM action (e.g., check-in, check-out, revision), with the option for users to enter detailed descriptions (adapted from Ref. [74]). (b) AUTODESK FUSION creates an automatically incremented version number for every file version saved and synced to the cloud. Unless the user otherwise provides a short description, only the timestamp is available to label the created versions. Users can also create milestones (based on the current version) with custom names and detailed descriptions, displayed with a flag sign. (c) ONSHAPE cascades all changes to the file under every user-created version, and branches are shown with additional visualization (i.e., a yellow branching path from the main blue branch). All three platforms display the user-specified version description with limited space in the history view, where additional clicks are required to open a separate dialogue window for a specific version to access the detailed documentation. All screenshots of software interfaces are presented for comparison purposes only.

files are “*all binary stuff, so it’s impossible to diff*” (RD22). Although THANGS recently introduced visual comparison of geometric differences between versions, as shown in Figure 5c [116], this feature still requires manual interpretation. In practice, users often rely on inconsistent user-generated descriptions to interpret changes, which vary widely in quality [26]. As a result, users need to constantly reconstruct the reasoning behind changes by manually examining all design parameters, which is an increasingly time-consuming process for large and interdependent models.

4.1.4 Branching and Merging Versions. As reviewed in Section 2.2.2, prior work has noted persistent shortcomings in branching workflows for CAD systems [22]. Our findings echo these concerns and highlight similar limitations in merging design branches, mentioned in 10 forum threads. Users frequently compared their expectations to software VCS, hoping for similar flexibility in CAD, such as “*a GitHub-like version control where you can create and fork different designs off of a base design and also merge back together different design branches*” (AD15).

In practice, however, merging remains technically challenging for CAD platforms like AUTODESK FUSION. Unlike text-based code, CAD models encode interdependent geometric and parametric data, making reconciliation across branches difficult. Yet users still expect

comparable flexibility and manageability in managing branches offered software VCS like GIT. As one SOLIDWORKS user put it, “*the feature-branch workflow with the audit/review step associated to a pull-request is a must. But they’re completely useless with CAD files ... (no merging capability)*” (SW13).

These experiences highlight a gap between users’ expectations, shaped by modern DVCS tools in software engineering, and the current technical constraints of CAD modelling, where interdependent geometric structures and modelling commands introduce complexities that make automatic merging far more difficult than in software contexts. As a result, while branching is sometimes supported, meaningful merge operations remain limited or absent across mainstream CAD platforms.

4.2 Continuity of Version Control in CAD

Version control in CAD needs to handle geometric data that are far more complex than the line-level textual edits of software development. Maintaining continuity across shared, linked, or modified models poses significant technical challenges.

4.2.1 Exporting Files with Version History. A recurring concern discovered in the forum threads is the loss of version history once a CAD file is exported or moved outside its original VCS-enabled

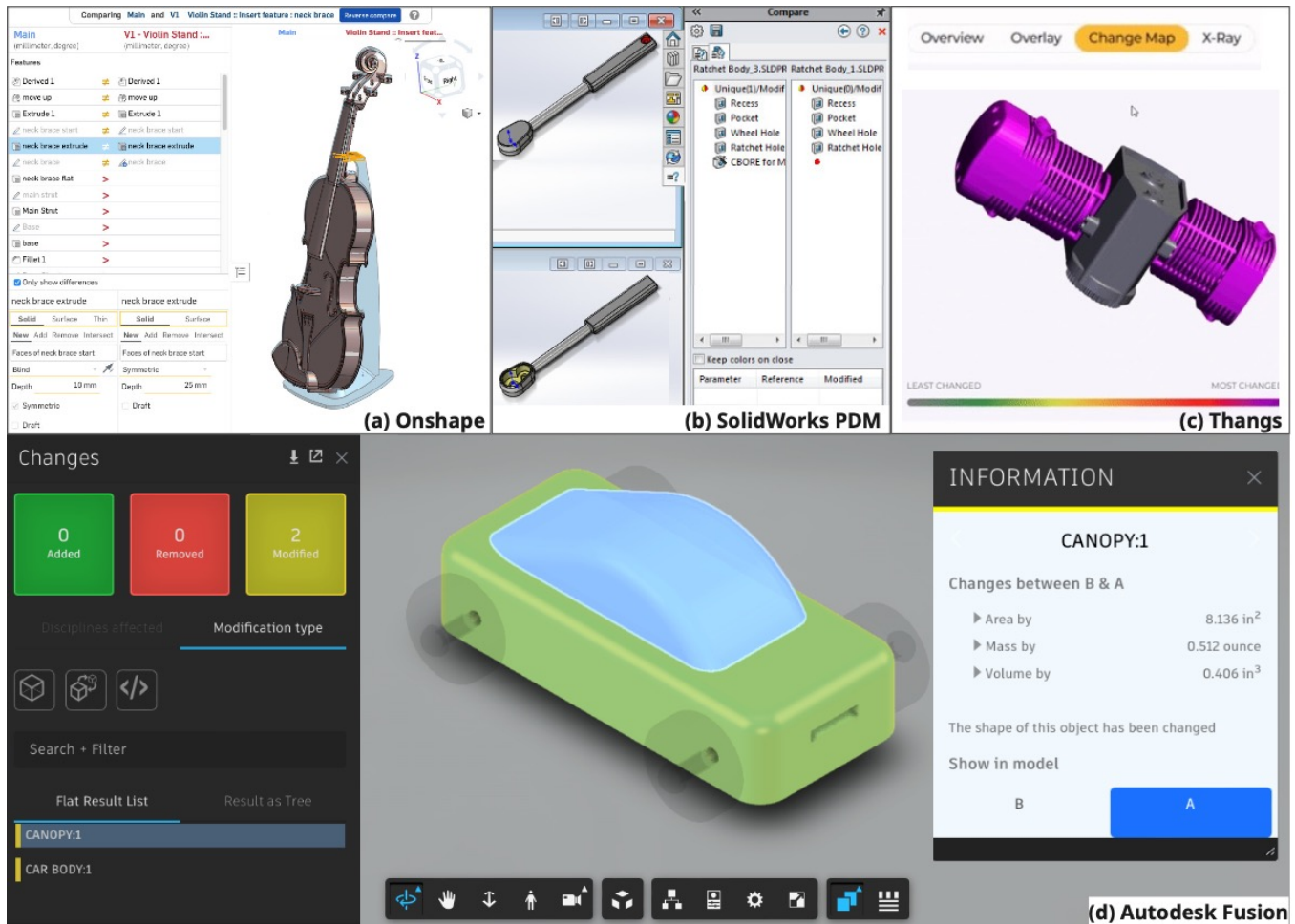


Figure 5: User interface for comparing two versions of a CAD model in the three studied CAD software and THINGS. (a) **ONSHAPE** presents two model versions with one version overlaid on top of another, where the geometric difference is highlighted in red. All modified modelling commands are listed in the side panel, where selecting each command presents all the modified command parameters, and the geometries that are modified by the selected command are highlighted in yellow. (b) **SOLIDWORKS** presents two versions of the model side by side, where changes of existing geometries are highlighted in yellow, and additions and deletions of geometries are highlighted in red. Modelling commands that were modified are also listed in the side panel, where selecting each command presents all the modified command parameters (adapted from Ref. [47]). (c) **THINGS** presents an overlaid view of two uploaded model versions, with geometric difference highlighted in purple (adapted from Ref. [116]). (d) **AUTODESK FUSION** presents only one version of the model that is under comparison with a list of modelling commands that are modified. However, selecting each listed modelling command only highlights the geometries that are modified by the selected command, but not the modified parameters. Modelling command parameters are not compared, even if they are accessible from the uploaded file format. All screenshots of software interfaces are presented for comparison purposes only.

workspace, mentioned in 19 forum threads. Unlike textual code that can be easily cloned and forked while preserving its full version history, modern CAD platforms only export the latest version of a model, where its prior versions become permanently unrecoverable, even if the model is later re-imported. This limitation restricts flexibility in file sharing and distribution of work. As one user noted, if “someone creates a design in their own project (rather than the parent one for the company), [they will have] issues getting the design moved over” (AD16). Such system rigidity effectively forces

teams to work within a single shared workspace, undermining the distributed collaboration that VCS is intended to support.

At the same time, re-importing files further compounds the problem, as the version context is often broken: “When multiple exported designs reference the same parts, those parts are imported multiple times and references are lost” (AD16). Since such references are not preserved across export-import cycles, teams must reconstruct these relationships manually. This lack of portability not only reduces design fluidity but also undermines collaborative workflows

and introduces duplicated work, especially across organizational boundaries and when model components are designed in multiple CAD platforms.

4.2.2 Propagating Changes to Linked Files. When dependencies are established between a base model and its variants (as described in Section 2.2.2), or when components are integrated into assemblies of parts, the task of propagating updates from the source model to all linked files remains a largely manual and error-prone process. This challenge was mentioned in 26 forum threads. As an ONSHAPE user described: *“Every modification I make (to a part [...]) in a separate document) requires a new version before I can see the changes in the assembly. This is very time consuming”* (OS20). After version creation, users also need to manually update every file that the modified part is used in (e.g., an assembly). This workflow can be especially confusing for beginners, because *“[the users] are not updating the [assembly itself], but the part ‘to’ the already updated [assembly]”* (OS19).

While some users appreciated this deliberate control so that they *“don’t have to worry about assemblies changing when [they]’ve made part changes that aren’t approved and released”* (OS19), others called for streamlined mechanisms, such as *“a simple button to push on the menu bar that will update the parts automatically, without having to select them all individually”* (OS19). These accounts illustrate the trade-off between maintaining deliberate control over design stability and approval workflow, and the need to reduce the manual overhead of keeping linked files synchronized. As designs grow in scale and complexity, and as teams vary in expertise and organizational practices, these competing needs become increasingly difficult to reconcile through current systems.

4.2.3 Updating Changes to the Source File. While Section 4.2.2 discussed propagating updates from a source model to its linked files, users also reported challenges in pushing changes made within a linked file back to the source model, mentioned in 11 forum threads. When components are inserted into an assembly, users often prefer to modify them *in-context* (i.e., within the assembly where the part model is inserted), instead of editing the source file. As one user explained: *“I want to keep all of my components and assemblies matched. If I update it within the assembly I would like for it to update component parts. Or if I update it within the base components I would like it to update in the assembly”* (AD15). Although in-context editing is now supported in all studied CAD modelling platforms, the workflow still requires users to *“create a version on every edit to see these updates in the assembly”* (OS21). Instead, an ONSHAPE user envisioned a more fluid workflow: *“My desired behavior would be to insert the part as the unversioned top of the branch so changes are reflected in real time, then apply a version once I’m finished”* (OS21). Such requests highlight users’ desire for workflows that preserve the benefits of versioning while supporting more fluid, real-time propagation of edits across assemblies and their source components.

4.3 Scope of Version Control in CAD

Version control preserves design history, yet CAD platforms differ in what can be versioned and at what granularity, ranging from

flexible tracking to rigid, system-imposed structures. These constraints often reflect necessary trade-offs that are not present in software engineering: given the complexity and interdependence of CAD models, CAD systems impose limitations to maintain stability and performance.

4.3.1 Versioning at Greater Granular Level. While ONSHAPE enables DVCS for CAD modelling, it organizes and versions data at a coarser granularity, which limits users from modular editing. This concern was mentioned in 21 forum threads. As shown in Figure 6, ONSHAPE restricts versioning to the document level, where a single document may contain multiple parts, assemblies, and drawings. In contrast, traditional file-based systems such as SOLIDWORKS and AUTODESK FUSION allow each of these files to be versioned independently. Users described this document-level structure as creating unnecessary versioning overhead, compounding the challenge of propagating changes to linked files (Section 4.2.2). For example, when users build *“a ‘library’ document for things like bolts, brackets, etc., and a new item is added to that library. It will cause an unnecessary version bump which can trigger a ‘newer version’ update request in a bunch of other documents where nothing has actually changed”* (OS16). In large CAD documents, this limitation further exacerbates other earlier challenges. For instance, identifying small localized changes becomes more difficult (Section 4.1.3), and version histories become increasingly cluttered with minor model updates (Section 4.1.1).

As a workaround, some users attempted to approximate file-level granularity by splitting large designs across multiple documents. Yet, this approach undermined ONSHAPE’s advantages: *“With the current setup, reverting to a previous [version of individual parts] would only be useful if I used a new document for every part, but this counteracts the convenience of creating multiple parts”* (OS18). Still, not all users viewed this constraint negatively. As one user reflected: *“From a data perspective, [this is] a small cost for the advantage of a very powerful tool”* (OS19). While some value precise control over individual components, others prefer the simplicity and cohesion of a unified workspace, even at the expense of granularity.

4.3.2 Untracking Unused Parts. In AUTODESK FUSION, users cannot delete any component that was once referenced in a past version of another file (e.g., an assembly that previously contained it), limiting users’ ability to clean up and organize workspaces over time. This concern was raised in 12 forum threads. This restriction preserves version integrity, ensuring earlier versions can be restored without broken references. However, as projects grow, assemblies evolve and certain parts become obsolete, and *“the ability to delete past designs and therefore old stale references [becomes crucial]. Meanwhile, [users’] design browsers are just landfills”* (AD15). In the absence of more flexible cleanup mechanisms, some users resort to improvised workarounds, such as *“placing [the unused files] in a bad files folder”* (AD16). While effective in the short term, these practices increase project complexity and waste storage space over time.

4.4 Distribution of Version Control in CAD

In collaborative CAD environments, cloud platforms distribute version histories across team members, enabling shared access to evolving designs. While this supports coordination, users raised

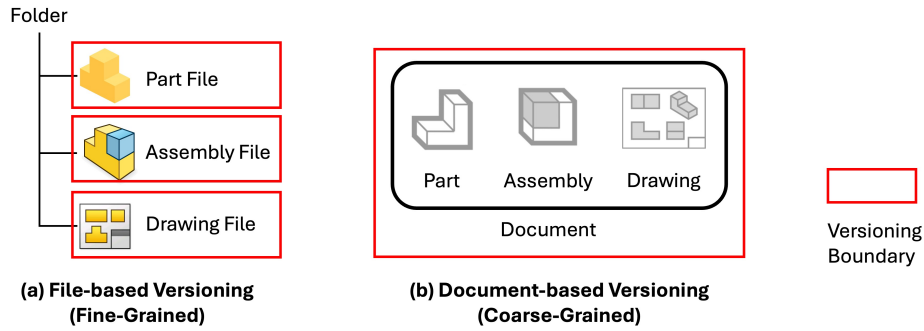


Figure 6: Comparison of versioning granularity across CAD platforms. (a) In file-based platforms like SOLIDWORKS and AUTODESK FUSION, each part, assembly, and drawing exists as a separate file, and changes are tracked independently at the file level. (b) In ONSHAPE, all components are grouped into a single document, and versioning applies to the entire document, where changes made in any component could create a new version for the whole document.

concerns about lacking fine-grained control over what aspects of the version history are shared and with whom.

4.4.1 Controlling User Access to Versions. When CAD models are shared with external collaborators, document owners often lack fine-grained control over the version history access, a concern mentioned in seven forum threads. In ONSHAPE, sharing a project reveals the entire version history [53], whereas at the other extreme, AUTODESK FUSION limits sharing access to a single version only [51]. This limitation presents practical and strategic challenges. For instance, a user explained that: “I am trying to share a version of a document to an outside vendor for quoting, along with download privileges so they can evaluate in their own CAD tool if needed. But I do not want the vendor to have access to the real-time document, only the ‘quote version’” (OS18).

To achieve more fine-grained sharing control, some users resort to cumbersome workarounds such as: “deriving the version of your part studio into a fresh document, or bringing the version of your main assembly into a blank assembly of a fresh document as a sub-assembly [...] Then share that fresh document to the vendor” (OS21). Even then, this approach remains flawed: “A lot of our quote processes are multi-round with tweaks made between rounds, so creating separate documents for each version would get out of hand” (OS21). Such user experiences highlight the need for scoped sharing, where version visibility can be tailored to the recipient’s role and the purpose of interaction.

4.5 Summary of Cross-Platform Challenges

Across the four thematic areas (i.e., management, continuity, scope, and distribution), we observed that certain challenges are universal across all three studied CAD platforms, while others are platform-specific. Table 3 synthesizes these findings, highlighting which user needs remain unmet and which challenges are partially addressed or fully supported. Although the three CAD platforms examined in this study, and many others not included, offer broadly similar technical capabilities for basic CAD modelling, they differ in their collaborative support mechanisms, including the critically important version control infrastructure.

In practice, users and organizations adopt and migrate between different CAD systems for diverse reasons, such as functional requirements (e.g., integration with downstream manufacturing tools), economical considerations (e.g., free or subsidized offerings for startups), industry standards (e.g., standardized format in exchanging off-the-shelf parts), operational considerations (e.g., operating system compatibility), and so on [60]. As engineers are trained and gain access to different CAD systems in both academic and professional settings, they experience the varying degrees of collaborative support these tools provide and discover a nuanced and uneven landscape of features, where features well-supported in one system may be absent or less refined in another. A prominent example is the DVCS architecture in ONSHAPE, which enables real-time collaboration but shifts version management from the file level to the document level. As a result, users transitioning from traditional, file-based CAD platforms (e.g., SOLIDWORKS, AUTODESK FUSION) can struggle to adapt to this coarser granularity of versioning in ONSHAPE, where individual parts and assemblies cannot be independently versioned (Section 4.3.1).

5 Design Opportunities

CAD has long served as a cornerstone technology in product development. Over the decades, its role has evolved from a standalone design tool to a complex socio-technical system that mediates collaboration among teams and organizations and supports the creation of increasingly intricate design artifacts. To sustain this shift toward large-scale, collaborative engineering design, version control has become a critical component of the underpinning *information infrastructure* in modern CAD systems. Similar to related fields such as software engineering [9, 32] and scientific collaboration [30, 50, 75, 86, 104], version control in CAD constitutes the backbone of coordination work. In line with Star and Ruhleder’s conception of infrastructure [106], version control operates largely in the background, quietly supporting collaboration, yet it both shapes and is shaped by user practices. More importantly, such infrastructures often becomes visible only when they break down, revealing the complexities of the collaborative work they support [48, 106]. Building on the empirical user challenges identified

Table 3: User challenges in CAD version control and the extent of support across studied commercial CAD platforms.

Challenge	Sec.	<i>n</i>	SOLIDWORKS	FUSION	ONSHAPE	Challenge type
Management of version control						
Navigating version history	4.1.1	47	◐	◐	◐	Universal
Editing version history	4.1.2	45	○	○	○	Universal
Identifying changes between versions	4.1.3	17	◐	◐	◐	Universal
Branching and merging versions	4.1.4	10	●	○	●	Platform specific
Continuity of version control						
Exporting files with version history	4.2.1	19	○	○	○	Universal
Propagating changes to linked files	4.2.2	26	●	◐	◐	Platform specific
Updating changes to the source file	4.2.3	11	●	●	●	Addressed
Scope of version control						
Versioning at greater granular level	4.3.1	21	●	●	○	Platform specific
Untracking unused parts	4.3.2	12	●	○	●	Platform specific
Distribution of version control						
Controlling user access to versions	4.4.1	7	●	◐	◐	Platform specific

Note: ● denotes that the challenge is fully resolved with all user needs observed in this study supported, ○ denotes that the challenge is currently unaddressed, and ◐ denotes that the challenge is partially resolved but with unmet user needs reported in this study for the specific CAD platform, as of August 2025.

in our study, this section discusses new research directions and design opportunities aimed at improving the support for collaborative design in CAD modelling platforms with version control.

5.1 Supporting the Articulation Work of the Design Process

Opportunity 1. During the product design process, users often create new model versions with inconsistent or incomplete change summaries (Sections 4.1.1 and 4.1.3). As versions accumulate, it becomes increasingly challenging to navigate version histories (Section 4.1.1), with significant design milestones obscured by minor updates (Section 4.1.2). These challenges suggest an opportunity to design tools that better support users in articulating the design history as a collaborative record. Specifically, such tools should (1) help users more consistently capture and describe version differences, (2) support users to organize and edit existing versions to reflect the evolving design process, and (3) present version histories in ways that are communicable to collaborators.

Throughout a product’s evolution, continuous effort is required to manage the design process as a shared collaborative record, one that extends beyond collaborators’ individual personal information spaces and integrates contributions from all team members (i.e., the articulation work) [12, 101, 111]. In fact, the ease of processing the history of a product may potentially influence the users’ impression of the design and ultimately influence work outcomes [79]. As evident in software engineering, version control effectively records the information necessarily and supports mechanisms in constructing a collaborative narrative [9]. However, due to the complexity of CAD models, a gap in supporting the articulation work of the design process remains unaddressed.

Capturing and describing version differences. When versions are created in CAD, users often label them with consistent but opaque numeric identifiers (Section 4.1.1) or with more descriptive but inconsistent, and potentially incomplete, summaries (Section 4.1.3), complicating how design histories are articulated and understood. As one CAD user noted in Section 4.1.3, effective version comparison should capture both the “what” (parametric/geometric differences) and the “why” (functional difference). Traditionally, such information is documented through formal engineering change notices or orders (ECNs/ECOs) [56, 121]. While the parametric differences may be algorithmically identified, the design knowledge that users leverage and their design intent are often tedious or challenging to capture [63]. As modern version control infrastructures streamline change procedures to support rapid and flexible iterations, they also reduce the reliance on structured documentation practices, leaving much contextual information unrecorded.

Rather than replacing the articulation needs, technology support should scaffold articulation work [78]. Without undermining the rapid design iterations enabled by modern CAD systems, there are opportunities to augment human productivity through closely supervised applications of artificial intelligence (AI). While existing rule-based comparison methods are already effective at detecting geometric differences effectively when models are compared in proprietary formats [14, 15] or even in neutral formats [77], the most laborious challenge remains the synthesis of both major and subtle design changes into meaningful yet concise documentation. Drawing on prior successes in automating code summarization using machine learning (ML) techniques in software engineering [31, 66, 70, 73, 125], future work may explore the use of large language models (LLMs) [88] and vision-language models (VLMs) [124] to assist in processing both the underlying code representation of CAD models and the visualized geometric differences,

such that candidate textual descriptions of the modifications can be generated.

Alternatively, extending the concept of think-aloud computing [63] offers a complementary approach, in which users verbalize their thoughts and observations while inspecting 3D geometric differences, or even during the entire modelling process, and the resulting speech is subsequently edited or potentially further summarized with LLM support into textual documentation. Prior work suggests that this approach can capture richer contextual information including the design intent, while reducing manual documentation effort [63]. Across these potential approaches, users must remain accountable for the version history entries they create, and the AI should only function as an assistive layer that enhances productivity without displacing human judgment. Standardized documentation templates may be one promising mechanism for structuring this interaction, simultaneously guiding ML models to structure its output and supporting users in reviewing and refining AI-assisted documentation.

Organizing and editing version histories. While immutability ensures traceability [28], current CAD systems provide limited mechanisms for editing or reorganizing records after the fact, degrading the navigability and interpretability of version histories as meaningless versions accumulate over time (Sections 4.1.1 and 4.1.2). In contrast, Git-based workflows in software engineering support retrospective cleanup and collaborative narrative construction, allowing contributors to co-construct a version history that serves not merely as a raw log of edits, but as a curated reflection of collective progress [9].

Although CAD geometries and the interdependence among model components are more complex than text-based source code, CAD systems like ONSHAPE have begun modernizing their version control support from CVCS to DVCS, recording incremental design changes when modelling operations are modified, rather than saving full model copies [5]. This shift opens opportunities for CAD systems with DVCS to implement more sophisticated Git-like operations that allow users to clean up and annotate version histories without compromising traceability. However, for CAD systems that remain CVCS-based and open-source environments where heterogeneous model formats are exchanged, DVCS-style functionality may not yet be technically feasible. In these cases, as users suggested in Section 4.1.2, lightweight curation tools such as customizable markers or tags could be used to foreground significant milestones and improve the interpretability of version histories while preserving the complete underlying record.

Presenting version histories. As a design project evolves, its version history inevitably grows, often becoming too extensive for users to comprehend and navigate effectively (Section 4.1.1). According to Dourish and Bellotti, effective collaboration support should assist users to construct both (1) a high-level awareness of the *character* of others' actions to help them structure their activities and avoid duplication of work, and (2) a low-level awareness of the *content* of those actions to allow fine-grained shared working and synergistic group behaviours [35]. However, the flat and linear format adopted by most commercial CAD platforms (see Figure 4a) often fails to reflect these layers of awareness, particularly in

collaborative settings where branching, merging, and parallel development are common [23]. As illustrated in Figure 7, CAD systems may draw inspiration from visualization techniques in software engineering [46, 89, 123], incorporating additional visual dimensions, markers, and colour encoding to represent design evolutions and enhance collaborative awareness within version histories.

5.2 Facilitating Cross-Boundary Collaboration

Opportunity 2. When users collaborate across multiple boundaries – whether they be parallel development paths within a project (Section 4.1.4), linked projects within a CAD system (Sections 4.2.2 and 4.2.3), or even elements across different CAD systems (Section 4.2.1) – technical constraints in the modern CAD ecosystem often lead to discontinuities in version control and poor interoperability. At the organizational level, these challenges are compounded by limited support for collaboration across functional roles, including insufficiently granular sharing and access control (Section 4.4.1). Together, these challenges highlight opportunities to (1) design mechanisms that support parallel product development and merge reconciliation, (2) design mechanisms that facilitate change propagation across linked projects within a system, (3) develop standards that enable seamless data exchange across proprietary platforms, and (4) design mechanisms that strengthen communication between design teams and external stakeholders.

Boundaries are inherent and fundamental to socio-technical design, supporting effective project management within teams and organizations, yet boundaries should also facilitate the sharing of knowledge and experience [27, 84]. Collaborative work around boundary objects often involves negotiating, redefining, and transforming the boundaries [20, 67], and such objects become critical components of the information infrastructure that mediate collaboration across teams and organizations [45]. In our context, each CAD model can operate as a boundary object, where it provides a shared reference through which design intent, manufacturability, and downstream considerations are coordinated [112]. However, as collaboration increasingly expands beyond any single project's predefined boundaries – across branches, linked projects, organizational roles, and software systems – our study identifies new forms of friction emerging at multiple boundary levels.

As summarized in Table 3 and discussed in Section 4.5, most challenges in cross-boundary collaboration within the same CAD platform appear to be platform-specific. Users report tedious and error-prone processes when propagating changes between linked projects (Sections 4.2.2 and 4.2.3), and they experience frustration with limited fine-grained sharing control when collaborating with external partners (Section 4.4.1). Similarly, while branching and merging are supported in systems such as SOLIDWORKS and ONSHAPE, they remain limited or absent in AUTODESK FUSION and most OSH platforms (Section 4.1.4). These challenges are not inherent technical constraints of version control infrastructures, but symptoms of uneven infrastructural maturity and diverging product design priorities across the CAD landscape. This suggests an opportunity for software providers to better support collaborative

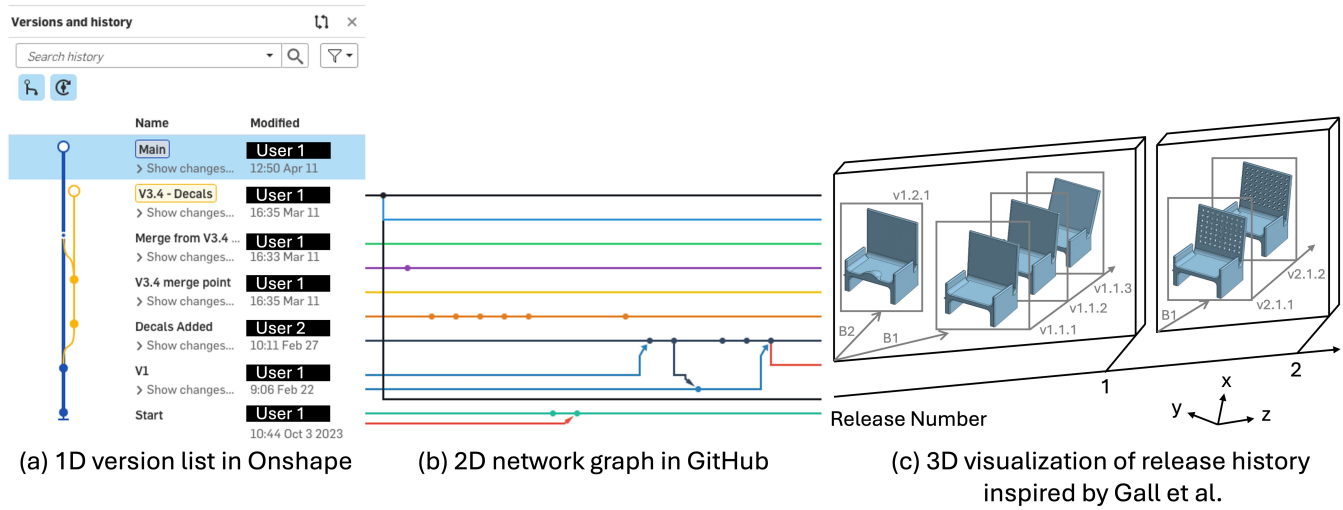


Figure 7: Sample version history visualizations with information presented in various dimensions. (a) ONSHAPE, as a representative of modern commercially available CAD software, presents the version history of a design project with a 1D graph, listing all versions in chronological order. (b) GITHUB, a software development platform with DVCS, presents the development history in the network view with branches of versions shown in parallel. (c) Inspired by Gall et al. [46], a third dimension can also be utilized to represent the release sequence number (RSN) of a product, where all versioning activities of a specific release are presented in a 2D graph, orthogonal to the direction that RSN increments. All screenshots of software interfaces are presented for comparison purposes only.

workflows by aligning functionality across platforms and recognizing cross-boundary collaboration as a core aspect of design work, rather than a peripheral feature. As Schmidt and Simone argued, distributed collaboration relies on constructing and running malleable and linkable computational coordination mechanisms [102], but such mechanisms remain underdeveloped in several CAD systems today.

In contrast, the challenges of cross-platform collaboration in CAD point to deeper infrastructural fragmentation that demands solutions and standards at the ecosystem level. Unlike textual code that can be readily read and edited in any text editor, all CAD platforms operate with different proprietary file formats. When CAD models are exchanged between systems, they are typically exported either in proprietary formats that are not interoperable when imported in other systems, or in standardized neutral formats such as STEP [41] that retain only static geometry while discarding version histories and other metadata [61] (Section 4.2.1). Once a model is exported, its associated version history is effectively lost, disrupting the continuity of the boundary object that mediates the shared understanding of design evolution. As design traceability and version control become increasingly vital for coordinating distributed work, CAD platforms may seek to build upon existing industry standards to enable bundled exports of multiple selected versions (e.g., milestones, releases) in interoperable formats and accept corresponding bundled imports that restore such data, akin to the “git bundle”¹² mechanism in software engineering. While these exported models would remain non-editable, such an approach would preserve essential historical checkpoints and capture

key design variants, allowing teams to revisit or restore specific design states even when collaborating across heterogeneous CAD environments. As the CAD ecosystem matures toward a more standardized and interconnected information infrastructure, it will be better positioned to sustain the continuity of CAD models as boundary objects, and hence support collaborative design work across organizational and technical boundaries.

From an HCI perspective, these discrepancies in collaboration support at the infrastructure level do not merely motivate calls for ecosystem-level standardization or isolated feature additions, but they also represent a rich site for systems research into how infrastructures are designed, appropriated, and stabilized over time. CAD systems are particularly revealing in this regard: once organizations commit to a specific platform, high switching costs (e.g., the manual reconstruction of designs in new proprietary formats) effectively lock in long-term use, allowing the consequences of infrastructural design decisions to unfold across years of collaborative practice. As we will further discuss in Section 5.3, users do not simply adapt to infrastructural constraints over time but actively reshape their workflows and develop workaround practices to sustain cross-boundary collaboration. This dynamic creates a distinctive opportunity for HCI systems research to study standards and interoperability mechanisms as socio-technical artifacts, shaped through use and negotiation rather than defined solely by technical specifications. As Hanseth and Lundberg argue, complex and domain-specific infrastructures should emerge by bridging communities of practice through standardization [48]. Rather than directly intervening by proposing a single “correct” infrastructure, HCI researchers can leverage established methods

¹²<https://git-scm.com/docs/git-bundle>

to analyze the diverging technical landscape and identify which coordination mechanisms warrant standardization in response to users' evolving collaborative needs. In this sense, standards and infrastructure design become an ongoing object of HCI inquiry for supporting cross-boundary collaboration in practice.

5.3 Designing Infrastructural Reflexivity

Opportunity 3. As each CAD system implements version control with distinct technical constraints, users frequently devise their own workarounds to approximate the collaborative workflows that exist in another CAD system (Sections 4.3.1, 4.3.2, and 4.4.1). These practices reveal not merely individual missing features but a broader need for CAD infrastructures that are designed to be reflexive to users and their uses, such that the system is capable of accommodating and learning from the diverse ways they are adapted and appropriated by users in practice [95].

Our study reveals how users negotiate infrastructure when the desired system support is lacking. Users creatively develop workarounds, such as organizing obsolete parts in “bad file folders” (Section 4.3.2), segmenting documents to approximate finer-grained versioning (Section 4.3.1), or duplicating documents to manage version sharing permissions (Section 4.4.1), thereby compensating for platform-level constraints. These practices exemplify what Pipek and Wulf describe as *infrastructuring*: the ongoing effort of aligning tools with local needs [95]. The infrastructural needs are often not recognized until breakdown [106]. We observe that rather than being passive recipients of technical constraints, users actively reconfigure their environments, effectively participating in the construction of their own version control infrastructure.

Similar to our discussion in Section 5.2, all the user challenges summarized in this section are also platform dependent. In fact, users often complain about the design workflow in one CAD system simply because they have experienced a more capable workflow in another system. This comparative frustration reflects the fact that while major CAD vendors compete to differentiate their products through distinct feature support, the CAD ecosystem lacks a standardized model of VCS. As a result, users must translate their practices across systems on their own to manually bridge the feature gaps. A move toward standardized version control as a shared information infrastructure across the CAD ecosystem would require maximizing flexibility, constantly asking whether one solution better supports future specific requirements than another, while recognizing that each solution both enables and constrains actions [49]. As Hanseth and Lundberg note, users will inevitably reshape a new infrastructure during use, and so such infrastructures “are, and should be, designed and implemented primarily by their users (and in use)” [48]. Taken together, these insights point toward an opportunity to develop CAD infrastructures that evolve with users rather than requiring users to adapt around infrastructural limitations.

Instead of attempting to anticipate all possible collaborative workflows upfront, we echo the argument that system designers should begin by articulating the principal interactions that the system is meant to support, independent of any particular workflow

prescription [37, 69]. Our findings suggest that one such interaction is the ability to treat design changes as something that can be selectively isolated across social and organizational contexts. Designing for this interaction implies defining a minimum yet expressive unit of versioning (e.g., a self-contained part, assembly, or drawing in Figure 6) and exposing operations over these units (e.g., versioning, branching, deleting, sharing) as composable primitives rather than fixed workflows. While such fine-grained abstractions may introduce technical trade-offs related to performance or security [38], our study highlights that overly coarse abstractions externalize this complexity onto users, who then reconstruct flexibility through brittle workarounds. Designing for infrastructural reflexivity hence shifts the design challenge from predicting use to enabling users to actively shape the infrastructure as part of their ongoing collaborative work.

5.4 Reflecting on the Design of VCS in CAD

Our study reveals fundamental limitations in how current CAD systems support version control. Rather than being simply another domain to which existing VCS paradigms can be applied, CAD exposes qualities and breakdowns of collaborative infrastructure that remain largely invisible in text-based environments, particularly within an ecosystem shaped by heterogeneous CAD platforms (Section 5.3) and proprietary file formats that lack interoperability (Section 5.2). These challenges reveal forms of articulation work that require collaborators to negotiate design intent through non-textual artifacts (Section 5.1). As cloud-based CAD like ONSHAPE increasingly blurs synchronous and asynchronous modes of work, emerging hybrid collaboration practices challenge the long-standing assumptions about concurrency, traceability, and version lineage in VCS design (Section 5.1), while simultaneously intensifying inconsistencies across the broader CAD ecosystem (Section 5.3). Taken together, these dynamics position version control in CAD as a *work-oriented infrastructure*: an infrastructure that supports highly complex and specialized practices whose properties are largely hidden for those who are not members of the community [48].

While CAD systems can learn from software development in offering more sophisticated VCS mechanisms, the interdependent and semantically rich nature of CAD artifacts require revisiting foundational VCS assumptions. For example, how should incremental changes be defined when enabling DVCS for CAD; how can version differences capture the implicit design intent beyond simple geometric comparison; and how might version histories be designed, managed, and presented to reflect hardware product development workflows. These questions signal the need to reconceptualize VCS in CAD as an evolving work-oriented infrastructure shaped through its users and in use, rather than being engineered top-down with assumptions [48]. As CAD vendors develop new versioning features and align functionality across platforms, the socio-technical dynamics in collaboration should remain central, as emphasized in other fields [1, 36, 97, 126]. New features or platform-level constraints should not disrupt the social coordination practices that design teams rely on [101]. Given the absence of a standardized or “correct” model for VCS in CAD, future systems should consider flexibility and reflexivity as core design principles, enabling infrastructures that adapt to diverse workflows, supporting ongoing

negotiation of meaning, and interoperate both within and across CAD platforms.

6 Limitations and Future Work

This paper identifies challenges in adopting version control for CAD-based mechanical product design, with findings shaped by the scope of our data sources. Our analysis is limited by its primary reliance on online forum discussions, which include a significantly greater proportion of issues of model creation in commercial CAD software than challenges of model sharing on open platforms like THINGIVERSE. To further build on these findings, future work may also adopt in-depth qualitative methods (e.g., interviews, contextual inquiry, observational studies) to examine how challenges manifest in different methods of collaboration (e.g., parallel editing, asynchronous collaboration), across industries (e.g., small-scale product design, large-scale engineering design), and within the workarounds users develop to cope with unmet needs. Such approaches would provide richer insight into the socio-technical dynamics of collaborative CAD and inform more grounded opportunities for system design.

Our scope was limited to three commercially available mechanical CAD platforms with built-in PDM solutions, which may not fully capture the diversity of practices across the broader CAD community. Other PDM/PLM solutions, such as PTC WINDCHILL and SIEMENS PLM, support cross-format version control primarily through centralized approaches. While we included SOLIDWORKS in our study to represent this traditional approach, our findings may not generalize to all enterprise workflows. At the same time, we acknowledge that the challenges and practices highlighted in this work are not exclusive to mechanical product design but may extend to other domains that also collaborate in comparable CAD environments (e.g., architecture, circuit design). Expanding future research to encompass third-party PLM solutions and additional design disciplines would provide a more comprehensive view of professional practices.

Finally, our analysis foregrounds user challenges while giving less attention to the technical and economic constraints faced by CAD software providers. Unlike source code, CAD data is geometrically complex and interdependent, complicating the direct transfer of Git-like features from software engineering. Addressing these challenges will require not only user-centred research but also collaboration with software vendors to understand feasibility and explore viable pathways for innovation.

7 Conclusion

In this paper, we presented an empirical study of user-reported challenges in applying version control to modern CAD software. Drawing on data mined from 424 threads across seven online forums, spanning both platform-specific and independent communities, we analyzed 170 relevant discussions to identify common pain points in CAD-based product design related to the management, continuity, scope, and distribution of versions. These findings enabled us to reflect on the design of state-of-the-art version control in CAD as an information infrastructure. The user challenges surfaced in our analysis further motivated design opportunities for future tools and mechanisms that support articulation work,

facilitate cross-boundary collaboration, and operate with infrastructural reflexivity. In general, version control in CAD is really the socially-complex, boundary-crossing work that is inevitable in large-scale collaboration for modern product design.

Our study delivers two sets of implications. For CAD software providers, it highlights opportunities to expand current infrastructures to better support collaborative workflows and the distributed, iterative nature of design work. For the HCI research community, it positions version control in CAD as a rich site for investigating socio-technical infrastructures, opening avenues for new tools, workflows, and standards to support collaborative product development. Ultimately, bridging insights from software engineering and HCI offers a promising path toward re-imagining CAD version control as an information infrastructure that better serves the realities of modern collaborative design practice.

Acknowledgments

We acknowledge the support of the Government of Canada's New Frontiers in Research Fund (NFRF) [NFRFE-2022-00543] and the Centre for Analytics and Artificial Intelligence Engineering (CARTE) at the University of Toronto. We are also grateful to our anonymous reviewers for their thoughtful and incisive comments.

References

- [1] Mostafa AbediniAla and Banani Roy. 2022. Facilitating Asynchronous Collaboration in Scientific Workflow Composition Using Provenance. *Proceedings of the ACM on Human-Computer Interaction* 6, EICS (June 2022), 1–26. <https://doi.org/10.1145/3534520>
- [2] Rafi Ahmed and Shamkant B. Navathe. 1991. Version management of composite objects in CAD databases. *ACM SIGMOD Record* 20, 2 (April 1991), 218–227. <https://doi.org/10.1145/119995.115825>
- [3] Celena Alcock, Nathaniel Hudson, and Parmit K. Chilana. 2016. Barriers to Using, Customizing, and Printing 3D Designs on Thingiverse. In *Proceedings of the 19th International Conference on Supporting Group Work*. ACM, Sanibel Island Florida USA, 195–199. <https://doi.org/10.1145/2957276.2957301>
- [4] Chukwuma M. Asuzu, Kathy Cheng, and Alison Olechowski. 2024. The Personas of Cloud CAD Collaboration: A Case Study of a Team of CAD Professionals. *IEEE Transactions on Engineering Management* 71 (2024), 11225–11237. <https://doi.org/10.1109/TEM.2024.3409178>
- [5] Ilya Baran. 2015. Under The Hood: How Collaboration Works. <https://www.onshape.com/en/blog/under-the-hood-how-collaboration-works>
- [6] Jeff Barrie. 2016. Applications for cloud-based CAD in design education and collaboration. In *DS 83: Proceedings of the 18th International Conference on Engineering and Product Design Education (E&PDE16), Design Education: Collaboration and Cross-Disciplinarity*. The Design Society, Aalborg, Denmark, 178–183.
- [7] Petr Baudiš. 2014. Current Concepts in Version Control Systems. <http://arxiv.org/abs/1405.3496> arXiv:1405.3496 [cs].
- [8] Gordon Baxter and Ian Sommerville. 2011. Socio-technical systems: From design methods to systems engineering. *Interacting with Computers* 23, 1 (Jan. 2011), 4–17. <https://doi.org/10.1016/j.intcom.2010.07.003>
- [9] Christian Bird, Peter C. Rigby, Earl T. Barr, David J. Hamilton, Daniel M. German, and Prem Devanbu. 2009. The promises and perils of mining git. In *2009 6th IEEE International Working Conference on Mining Software Repositories*. IEEE, Vancouver, BC, Canada, 1–10. <https://doi.org/10.1109/MSR.2009.5069475>
- [10] B.E. Biçici and C. Cangelir. 2012. Collaborative Digital Data Management for Design and Production. *Procedia CIRP* 3 (2012), 585–590. <https://doi.org/10.1016/j.procir.2012.07.100>
- [11] Guy André Boy. 2017. Human-centered design of complex systems: An experience-based approach. *Design Science* 3 (2017), e8. <https://doi.org/10.1017/dsj.2017.8>
- [12] LouAnne E. Boyd, Kyle Rector, Halley Profita, Abigale J. Stangl, Annuska Zolyomi, Shaun K. Kane, and Gillian R. Hayes. 2017. Understanding the Role Fluidity of Stakeholders During Assistive Technology Research "In the Wild". In *Proceedings of the 2017 CHI Conference on Human Factors in Computing Systems*. ACM, Denver Colorado USA, 6147–6158. <https://doi.org/10.1145/3025453.3025493>
- [13] Caius Brindescu, Mihai Codoban, Sergii Shmarkatiuk, and Danny Dig. 2014. How do centralized and distributed version control systems impact software changes?. In *Proceedings of the 36th International Conference on Software Engineering*. ACM,

- Hyderabad India, 322–333. <https://doi.org/10.1145/2568225.2568322>
- [14] Antoine Brière-Côté, Louis Rivest, and Roland Maranzana. 2012. Comparing 3D CAD Models: Uses, Methods, Tools and Perspectives. *Computer-Aided Design and Applications* 9, 6 (Jan. 2012), 771–794. <https://doi.org/10.3722/cadaps.2012.771-794>
- [15] Antoine Brière-Côté, Louis Rivest, and Roland Maranzana. 2013. 3D CAD Model Comparison: An Evaluation of Model Difference Identification Technologies. *Computer-Aided Design and Applications* 10, 2 (Jan. 2013), 173–195. <https://doi.org/10.3722/cadaps.2013.173-195>
- [16] Zack Brown. 2018. A Git Origin Story. <https://www.linuxjournal.com/content/git-origin-story>
- [17] Erin Buehler, Stacy Branham, Abdullah Ali, Jeremy J. Chang, Megan Kelly Hoffmann, Amy Hurst, and Shaun K. Kane. 2015. Sharing is Caring: Assistive Technology Designs on Thingiverse. In *Proceedings of the 33rd Annual ACM Conference on Human Factors in Computing Systems*. ACM, Seoul Republic of Korea, 525–534. <https://doi.org/10.1145/2702123.2702525>
- [18] Jorge D. Camba, Manuel Contero, and Pedro Company. 2016. Parametric CAD modeling: An analysis of strategies for design reusability. *Computer-Aided Design* 74 (May 2016), 18–31. <https://doi.org/10.1016/j.cad.2016.01.003> Publisher: Elsevier Ltd.
- [19] Pascale Carayon. 2006. Human factors of complex sociotechnical systems. *Applied Ergonomics* 37, 4 (July 2006), 525–535. <https://doi.org/10.1016/j.apergo.2006.04.011>
- [20] Paul R. Carile. 2004. Transferring, Translating, and Transforming: An Integrative Framework for Managing Knowledge Across Boundaries. *Organization Science* 15, 5 (Oct. 2004), 555–568. <https://doi.org/10.1287/orsc.1040.0094>
- [21] Zhiyong Chang, Jie Zhao, and Rong Mo. 2007. A Study of Version Control for Collaborative CAD. In *Cooperative Design, Visualization, and Engineering*. Springer Berlin Heidelberg, Shanghai, China, 140–148. https://doi.org/10.1007/978-3-540-74780-2_16
- [22] Kathy Cheng, Phil Cuvin, Alison Olechowski, and Shurui Zhou. 2023. User Perspectives on Branching in Computer-Aided Design. *Proceedings of the ACM on Human-Computer Interaction* 7, CSCW2 (Sept. 2023), 1–30. <https://doi.org/10.1145/3610220>
- [23] Kathy Cheng, Michal K. Davis, Xiyue Zhang, Shurui Zhou, and Alison Olechowski. 2023. In the Age of Collaboration, the Computer-Aided Design Ecosystem is Behind: An Interview Study of Distributed CAD Practice. *Proceedings of the ACM on Human-Computer Interaction* 7, CSCW1 (April 2023), 1–29. <https://doi.org/10.1145/3579613>
- [24] Kathy Cheng and Alison Olechowski. 2024. Analysis of Collaborative Assembly in Multi-User Computer-Aided Design. *Journal of Mechanical Design* 146, 3 (March 2024), 031701. <https://doi.org/10.1115/1.4063759>
- [25] Kathy Cheng, Alison Olechowski, and Shurui Zhou. 2025. It's a Complete Haystack: Understanding Dependency Management Needs in Computer-Aided Design. *Proceedings of the ACM on Human-Computer Interaction* 9, 7 (Oct. 2025), 1–32. <https://doi.org/10.1145/3757617>
- [26] Kathy Cheng, Shurui Zhou, and Alison Olechowski. 2024. "A Lot of Moving Parts": A Case Study of Open-Source Hardware Design Collaboration in the Thingiverse Community. *Proceedings of the ACM on Human-Computer Interaction* 8, CSCW2 (Nov. 2024), 1–29. <https://doi.org/10.1145/3687008>
- [27] Albert Chens. 1976. The Principles of Sociotechnical Design. *Human Relations* 29, 8 (Aug. 1976), 783–792. <https://doi.org/10.1177/001872677602900806>
- [28] Hong-Tai Chou and Won Kim. 1986. A Unifying Framework for Version Control in a CAD Environment. In *Proceedings of the 12th International Conference on Very Large Data Bases (VLDB '86)*. Morgan Kaufmann Publishers Inc., San Francisco California USA, 336–344. <https://doi.org/10.5555/645913.671319>
- [29] Dexin Chu, Xuening Chu, Yupeng Li, Guolin Lyu, and Deyi Xue. 2016. A multi-skeleton modelling approach based on top-down design and modular product design for development of complex product layouts. *Journal of Engineering Design* 27, 10 (Oct. 2016), 725–750. <https://doi.org/10.1080/09544828.2016.1227428> Publisher: Informa UK Limited.
- [30] Antonio Correia, Shoaib Jameel, Daniel Schneider, Benjamim Fonseca, and Hugo Paredes. 2019. The Effect of Scientific Collaboration on CSCW Research: A Scientometric Study. In *2019 IEEE 23rd International Conference on Computer Supported Cooperative Work in Design (CSCWD)*. IEEE, Porto, Portugal, 129–134. <https://doi.org/10.1109/CSCWD.2019.8791855>
- [31] Luis Fernando Cortes-Coy, Mario Linares-Vasquez, Jairo Aponte, and Denys Poshyvanyk. 2014. On Automatically Generating Commit Messages via Summarization of Source Code Changes. In *2014 IEEE 14th International Working Conference on Source Code Analysis and Manipulation*. IEEE, Victoria, BC, Canada, 275–284. <https://doi.org/10.1109/SCAM.2014.14>
- [32] Laura Dabbish, Colleen Stuart, Jason Tsay, and Jim Herbsleb. 2012. Social coding in GitHub: transparency and collaboration in an open software repository. In *Proceedings of the ACM 2012 conference on Computer Supported Cooperative Work*. ACM, Seattle Washington USA, 1277–1286. <https://doi.org/10.1145/2145204.2145396>
- [33] Yuanzhe Deng, Tucker Marion, and Alison Olechowski. 2022. Does Synchronous Collaboration Improve Collaborative Computer-Aided Design Output: Results From a Large-Scale Competition. In *Volume 6: 34th International Conference on Design Theory and Methodology (DTM)*. American Society of Mechanical Engineers, St. Louis, Missouri, USA, V006T06A026. <https://doi.org/10.1115/DETC2022-89731>
- [34] Yuanzhe Deng, Matthew Mueller, Chris Rogers, and Alison Olechowski. 2022. The multi-user computer-aided design collaborative learning framework. *Advanced Engineering Informatics* 51 (Jan. 2022), 101446. <https://doi.org/10.1016/j.aei.2021.101446>
- [35] Paul Dourish and Victoria Bellotti. 1992. Awareness and coordination in shared workspaces. In *Proceedings of the 1992 ACM conference on Computer-supported cooperative work - CSCW '92*. ACM Press, Toronto, Ontario, Canada, 107–114. <https://doi.org/10.1145/143457.143468>
- [36] Paul Edwards, Geoffrey Bowker, Steven Jackson, and Robin Williams. 2009. Introduction: An Agenda for Infrastructure Studies. *Journal of the Association for Information Systems* 10, 5 (May 2009), 364–374. <https://doi.org/10.17705/1jais.00200>
- [37] W. Keith Edwards, Victoria Bellotti, Anind K. Dey, and Mark W. Newman. 2003. The challenges of user-centered design and evaluation for infrastructure. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. ACM, Ft. Lauderdale Florida USA, 297–304. <https://doi.org/10.1145/642611.642664>
- [38] W. Keith Edwards, Mark W. Newman, and Erika Shehan Poole. 2010. The infrastructure problem in HCI. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. ACM, Atlanta Georgia USA, 423–432. <https://doi.org/10.1145/1753326.1753390>
- [39] Jacky Estublier and Rubby Casallas. 1995. Three dimensional versioning. In *Software Configuration Management*, Gerhard Goos, Juris Hartmanis, Jan Leeuwen, and Jacky Estublier (Eds.). Vol. 1005. Springer Berlin Heidelberg, Berlin, Heidelberg, 118–135. https://doi.org/10.1007/3-540-60578-9_9 Series Title: Lecture Notes in Computer Science.
- [40] Keenan Eves, John Salmon, Jacob Olsen, and Fred Fagergren. 2018. A comparative analysis of computer-aided design team performance with collaboration software. *Computer-Aided Design and Applications* 15, 4 (2018), 476–487. <https://doi.org/10.1080/16864360.2017.1419649>
- [41] International Organization for Standardization (ISO). 2002. STEP-file, ISO 10303-21 – Industrial automation systems and integration – Product data representation and exchange – Part 21: Implementation methods: Clear text encoding of the exchange structure. <https://www.iso.org/standard/33713.html>
- [42] Jessie Frazzelle. 2021. A New Era for Mechanical CAD: Time to move forward from decades-old design. *Queue* 19, 2 (April 2021), 5–16. <https://doi.org/10.1145/3466132.3469844>
- [43] David J. French, Brett Stone, Thomas T. Nysetvold, Ammon Hepworth, and W. Edward Red. 2016. Collaborative Design Principles From Minecraft With Applications to Multi-User Computer-Aided Design. *Journal of Computing and Information Science in Engineering* 16, 2 (June 2016), 1–9. <https://doi.org/10.1115/1.4032667>
- [44] Autodesk Fusion. 2024. Milestones and versions. <https://help.autodesk.com/view/fusion360/ENU/?guid=TPD-DESIGN-MILESTONES>
- [45] Uri Gal, Kalle Lyytinen, and Youngjin Yoo. 2008. The dynamics of IT boundary objects, information infrastructures, and organisational identities: the introduction of 3D modelling technologies into the architecture, engineering, and construction industry. *European Journal of Information Systems* 17, 3 (June 2008), 290–304. <https://doi.org/10.1057/ejis.2008.13> Publisher: Informa UK Limited.
- [46] H. Gall, M. Jazayeri, and C. Riva. 1999. Visualizing software release histories: the use of color and third dimension. In *Proceedings IEEE International Conference on Software Maintenance - 1999 (ICSM'99)*. 'Software Maintenance for Business Change' (Cat. No.99CB36360). IEEE, Oxford, UK, 99–108. <https://doi.org/10.1109/ICSM.1999.792584>
- [47] GoEngineer. 2017. Comparing SOLIDWORKS PDM File Versions. <https://www.goengineer.com/blog/comparing-solidworks-pdm-file-versions>
- [48] Ole Hanseth and Nina Lundberg. 2001. Designing Work Oriented Infrastructures. *Computer Supported Cooperative Work (CSCW)* 10, 3-4 (Sept. 2001), 347–372. <https://doi.org/10.1023/A:1012727708439>
- [49] Ole Hanseth, Eric Monteiro, and Morten Hatling. 1996. Developing Information Infrastructure: The Tension Between Standardization and Flexibility. *Science, Technology, & Human Values* 21, 4 (Oct. 1996), 407–426. <https://doi.org/10.1177/016224399602100402>
- [50] Noriko Hara, Paul Solomon, Seung-Lye Kim, and Diane H. Sonnenwald. 2003. An emerging view of scientific collaboration: Scientists' perspectives on collaboration and factors that impact collaboration. *Journal of the American Society for Information Science and Technology* 54, 10 (Aug. 2003), 952–965. <https://doi.org/10.1002/asi.10291>
- [51] Fusion Help. 2025. Share designs from the Fusion web client. <https://help.autodesk.com/view/fusion360/ENU/?guid=GUID-1500941F-6EB7-4058-8CD9-7642DEA250E7>
- [52] Onshape Help. 2024. Versions and History. <https://cad.onshape.com/help/Content/versionmanager.htm>

- [53] Onshape Help. 2025. Share Documents. <https://cad.onshape.com/help/Content/shareddocuments.htm>
- [54] Bryce Heventhal. 2018. Fusion 360 Mastery: How to Use Version Control to Easily Manage Your Design Projects. <https://www.autodesk.com/products/fusion-360/blog/fusion-360-mastery-manage-design-projects/>
- [55] Megan Hofmann, Gabriella Hann, Scott E. Hudson, and Jennifer Mankoff. 2018. Greater than the Sum of its PARTs: Expressing and Reusing Design Intent in 3D Models. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems (CHI '18)*. Association for Computing Machinery, New York, NY, USA, 1–12. <https://doi.org/10.1145/3173574.3173875>
- [56] T. A. W. Jarratt, C. M. Eckert, N. H. M. Caldwell, and P. J. Clarkson. 2011. Engineering change: an overview and perspective on the literature. *Research in Engineering Design* 22, 2 (April 2011), 103–124. <https://doi.org/10.1007/s00163-010-0097-y>
- [57] Qiang Ji and Michael M. Marefat. 1997. Machine interpretation of CAD data for manufacturing applications. *Comput. Surveys* 29, 3 (Sept. 1997), 264–311. <https://doi.org/10.1145/262009.262012>
- [58] Stefan Junk and Christian Kuen. 2016. Review of Open Source and Freeware CAD Systems for Use with 3D-Printing. *Procedia CIRP* 50 (2016), 430–435. <https://doi.org/10.1016/j.procir.2016.04.174> 26th CIRP Design Conference.
- [59] Stefan Junk and Dennis Spannbauer. 2018. Use of Cloud-Based Computer Aided Design Software in Design Education. In *2018 17th International Conference on Information Technology Based Higher Education and Training (ITHET)*. IEEE, Olhao, 1–6. <https://doi.org/10.1109/ITHET.2018.8424769>
- [60] G. Kannan and V. P. Vinay. 2008. Multi-criteria decision making for the selection of CAD/CAM system. *International Journal on Interactive Design and Manufacturing (IJDeM)* 2, 3 (Aug. 2008), 151–159. <https://doi.org/10.1007/s12008-008-0045-5>
- [61] D.J. Kasik, W. Buxton, and D.R. Ferguson. 2005. Ten CAD Challenges. *IEEE Computer Graphics and Applications* 25, 2 (March 2005), 81–92. <https://doi.org/10.1109/MCG.2005.48>
- [62] Karthik Krishnamurthy and Kincho H. Law. 1995. Version Management in a CAD Paradigm. In *ASME 1995 15th International Computers in Engineering Conference and the ASME 1995 9th Annual Engineering Database Symposium*. American Society of Mechanical Engineers, Boston, Massachusetts, USA, 1133–1144. <https://doi.org/10.1115/EDM1995-0845>
- [63] Rebecca Krosnick, Fraser Anderson, Justin Matejka, Steve Oney, Walter S. Lasecki, Tovi Grossman, and George Fitzmaurice. 2021. Think-Aloud Computing: Supporting Rich and Low-Effort Knowledge Capture. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*. ACM, Yokohama Japan, 1–13. <https://doi.org/10.1145/3411764.3445066>
- [64] Mike La Fleche. 2019. Gitflow Version Management. <https://learn.onshape.com/learn/article/gitflow-version-management>
- [65] Patricia Lago, Henry Muccini, and Hans Van Vliet. 2009. A scoped approach to traceability management. *Journal of Systems and Software* 82, 1 (Jan. 2009), 168–182. <https://doi.org/10.1016/j.jss.2008.08.026>
- [66] Alexander LeClair, Sakib Haque, Lingfei Wu, and Collin McMillan. 2020. Improved Code Summarization via a Graph Neural Network. In *Proceedings of the 28th International Conference on Program Comprehension*. ACM, Seoul Republic of Korea, 184–195. <https://doi.org/10.1145/3387904.3389268>
- [67] Charlotte P. Lee. 2007. Boundary Negotiating Artifacts: Unbinding the Routine of Boundary Objects and Embracing Chaos in Collaborative Work. *Computer Supported Cooperative Work (CSCW)* 16, 3 (June 2007), 307–339. <https://doi.org/10.1007/s10606-007-9044-5>
- [68] Forrest Leighton. 2018. MakerBot Thingiverse Celebrates 10 Years of 3D Printed Things. <https://investors.stratasys.com/news-events/press-releases/detail/547/makerbot-thingiverse-celebrates-10-years-of-3d-printed>
- [69] Jingyi Li, Eric Rawn, Jacob Ritchie, Jasper Tran O'Leary, and Sean Follmer. 2023. Beyond the Artifact: Power as a Lens for Creativity Support Tools. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*. ACM, San Francisco CA USA, 1–15. <https://doi.org/10.1145/3586183.3606831>
- [70] Mario Linares-Vasquez, Luis Fernando Cortes-Coy, Jairo Aponte, and Denys Poshyvanyk. 2015. ChangeScribe: A Tool for Automatically Generating Commit Messages. In *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*. IEEE, Florence, Italy, 709–712. <https://doi.org/10.1109/ICSE.2015.229>
- [71] Lukas Linsbauer, Thorsten Berger, and Paul Grünbacher. 2017. A classification of variation control systems. In *Proceedings of the 16th ACM SIGPLAN International Conference on Generative Programming: Concepts and Experiences*. ACM, Vancouver BC Canada, 49–62. <https://doi.org/10.1145/3136040.3136054>
- [72] Lung-Chun Liu. 1990. Design data management in a CAD framework environment. In *Conference proceedings on 27th ACM/IEEE design automation conference - DAC '90*. ACM Press, Orlando, Florida, United States, 156–161. <https://doi.org/10.1145/123186.123245>
- [73] Zhongxin Liu, Xin Xia, Ahmed E. Hassan, David Lo, Zhenchang Xing, and Xinyu Wang. 2018. Neural-machine-translation-based commit message generation: how far we are?. In *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering*. ACM, Montpellier France, 373–384. <https://doi.org/10.1145/3238147.3238190>
- [74] Samir Lohmann. 2018. Can I get a new revision number for a file in SOLIDWORKS PDM prior to making my changes? <https://www.javelin-tech.com/blog/2018/10/solidworks-pdm-revision-number-workflow/> Section: SOLIDWORKS PDM.
- [75] Shiyong Lu and Jia Zhang. 2011. Collaborative scientific workflows supporting collaborative science. *International Journal of Business Process Integration and Management* 5, 2 (2011), 185. <https://doi.org/10.1504/IJBPM.2011.040209>
- [76] Thomas Ludwig, Oliver Stickel, Alexander Boden, and Volkmar Pipek. 2014. Towards sociable technologies: an empirical study on designing appropriation infrastructures for 3D printing. In *Proceedings of the 2014 conference on Designing interactive systems*. ACM, Vancouver BC Canada, 835–844. <https://doi.org/10.1145/2598510.2598528>
- [77] Hao Lv, Jin Cheng, Zhenyu Liu, Weifei Hu, and Jianrong Tan. 2024. Difference identification of 3D CAD models based on key-point matching oriented to engineering change management. *Advanced Engineering Informatics* 62 (Oct. 2024), 102595. <https://doi.org/10.1016/j.aei.2024.102595>
- [78] Gloria Mark, Victor Gonzalez, Marcello Sarini, and Carla Simone. 2002. Supporting articulation with the reconciler. In *CHI '02 Extended Abstracts on Human Factors in Computing Systems*. ACM, Minneapolis Minnesota USA, 814–815. <https://doi.org/10.1145/506443.506611>
- [79] Jennifer Marlow and Laura A. Dabbish. 2015. The Effects of Visualizing Activity History on Attitudes and Behaviors in a Peer Production Context. In *Proceedings of the 18th ACM Conference on Computer Supported Cooperative Work & Social Computing*. ACM, Vancouver BC Canada, 757–764. <https://doi.org/10.1145/2675133.2675250>
- [80] Salome Maro, Anthony Anjorin, Rebekka Wohlrab, and Jan-Philipp Steghöfer. 2016. Traceability maintenance: factors and guidelines. In *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering*. ACM, Singapore Singapore, 414–425. <https://doi.org/10.1145/2970276.2970314>
- [81] Mahmoud Masmoudi, Patrice Leclaire, Marc Zolghadri, and Mohamed Haddar. 2015. Dependency identification for engineering change management (ECM): an example of computer-aided design (CAD)-based approach. In *Proceedings of the 20th International Conference on Engineering Design (ICED 15) (ICED, Vol. 3)*. The Design Society, Milan, Italy, 199–208.
- [82] Tommi Mikkonen and Antti Nieminen. 2012. Elements for a cloud-based development environment: online collaboration, revision control, and continuous integration. In *Proceedings of the WICSA/ECSA 2012 Companion Volume*. ACM, Helsinki Finland, 14–20. <https://doi.org/10.1145/2361999.2362003>
- [83] William J Mitchell. 1995. CAD as a Social Process. In *Sixth International Conference on Computer-Aided Architectural Design Futures*. CAAD Futures, Singapore, 7–9. <https://papers.cumincad.org/cgi-bin/works/paper/f766>
- [84] Enid Mumford. 2006. The story of socio-technical design: reflections on its successes, failures and potential. *Information Systems Journal* 16, 4 (Oct. 2006), 317–342. <https://doi.org/10.1111/j.1365-2575.2006.00221.x>
- [85] Institute of Education Science. 2017. *Reviewer Guidance for Use with the Procedures and Standards Handbooks*. Technical Report. Institute of Education Science. https://ies.ed.gov/ncee/WWC/Docs/ReferenceResources/www_reviewer_guidance_103017.pdf
- [86] Agnieszka Olechnicka, Adam Ploszaj, and Dorota Celińska-Janowicz. 2018. *The Geography of Scientific Collaboration* (1 ed.). Routledge, London. <https://doi.org/10.4324/9781315471938>
- [87] Onshape. 2024. Eliminating CAD Version Control Issues with the Cloud-Native Software. <https://www.onshape.com/en/blog/a-single-source-of-truth-eliminating-version-control-issues-with-the-cloud>
- [88] Rock Yuren Pang, Hope Schroeder, Kynneddy Simone Smith, Solon Barocas, Ziang Xiao, Emily Tseng, and Danielle Bragg. 2025. Understanding the LLM-ification of CHI: Unpacking the Impact of LLMs at CHI through a Systematic Literature Review. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*. ACM, Yokohama Japan, 1–20. <https://doi.org/10.1145/3706598.3713726>
- [89] Jungkook Park, Yeong Hoon Park, Suin Kim, and Alice Oh. 2017. Eliph: Effective Visualization of Code History for Peer Assessment in Programming Education. In *Companion of the 2017 ACM Conference on Computer Supported Cooperative Work and Social Computing*. ACM, Portland Oregon USA, 33–36. <https://doi.org/10.1145/3022198.3023266>
- [90] Vrushank Phadnis, Hamza Arshad, David Wallace, and Alison Olechowski. 2021. Are Two Heads Better Than One for Computer-Aided Design? *Journal of Mechanical Design* 143, 7 (July 2021), 1–38. <https://doi.org/10.1115/1.4050734>
- [91] Vrushank S. Phadnis, David R. Wallace, and Alison Olechowski. 2021. A multimodal experimental approach to study CAD collaboration. *Computer-Aided Design and Applications* 18, 2 (2021), 328–342. <https://doi.org/10.14733/cadaps.2021.328-342>
- [92] Shaun Phillips, Jonathan Sillito, and Rob Walker. 2011. Branching and merging: an investigation into current version control practices. In *Proceedings of the 4th International Workshop on Cooperative and Human Aspects of Software Engineering*. ACM, Waikiki, Honolulu HI USA, 9–15. <https://doi.org/10.1145/1984642.1984645>

- [93] Physna. 2020. Introducing Thangs: The World's Most Powerful Geometric Search Engine and Collaboration Platform for 3D Models. <https://www.prnewswire.com/news-releases/introducing-thangs-the-worlds-most-powerful-geometric-search-engine-and-collaboration-platform-for-3d-models-301113959.html>
- [94] Les A. Piegl. 2005. Ten challenges in computer-aided design. *Computer-Aided Design* 37, 4 (April 2005), 461–470. <https://doi.org/10.1016/j.cad.2004.08.012>
- [95] Volkmar Pipek and Volker Wulf. 2009. Infrastructuring: Toward an Integrated Perspective on the Design and Use of Information Technology. *Journal of the Association for Information Systems* 10, 5 (May 2009), 447–473. <https://doi.org/10.17705/1jais.00195>
- [96] Rahul Premraj, Antony Tang, Nico Linssen, Hub Geraats, and Hans Van Vliet. 2011. To branch or not to branch?. In *Proceedings of the 2011 International Conference on Software and Systems Process*. ACM, Waikiki, Honolulu HI USA, 81–90. <https://doi.org/10.1145/1987875.1987890>
- [97] Eric Rawn, Jingyi Li, Eric Paulos, and Sarah E. Chasins. 2023. Understanding Version Control as Material Interaction with Quickpose. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*. ACM, Hamburg Germany, 1–18. <https://doi.org/10.1145/3544548.3581394>
- [98] D. Robertson and T.J. Allen. 1993. CAD system use and engineering performance. *IEEE Transactions on Engineering Management* 40, 3 (1993), 274–282. <https://doi.org/10.1109/17.233189>
- [99] Nayan B. Ruparelia. 2010. The history of version control. *ACM SIGSOFT Software Engineering Notes* 35, 1 (Jan. 2010), 5–9. <https://doi.org/10.1145/1668862.1668876>
- [100] Johnny Saldaña. 2013. *The coding manual for qualitative researchers* (2. ed ed.). SAGE Publ, Los Angeles, Calif.
- [101] Kjeld Schmidt and Liam Bannon. 1992. Taking CSCW seriously: Supporting articulation work. *Computer Supported Cooperative Work (CSCW)* 1, 1-2 (March 1992), 7–40. <https://doi.org/10.1007/BF00752449>
- [102] Kjeld Schmidt and Carla Simonee. 1996. Coordination mechanisms: Towards a conceptual foundation of CSCW systems design. *Computer Supported Cooperative Work (CSCW)* 5, 2-3 (1996), 155–200. <https://doi.org/10.1007/BF00133655>
- [103] SOLIDWORKS. 2019. SOLIDWORKS PDM. <https://www.solidworks.com/product/solidworks-pdm>
- [104] Diane H. Sonnenwald. 2007. Scientific collaboration. *Annual Review of Information Science and Technology* 41, 1 (Jan. 2007), 643–681. <https://doi.org/10.1002/aris.2007.1440410121>
- [105] Susan Leigh Star and Karen Ruhleder. 1994. Steps towards an ecology of infrastructure: complex problems in design and access for large-scale collaborative systems. In *Proceedings of the 1994 ACM conference on Computer supported cooperative work - CSCW '94*. ACM Press, Chapel Hill, North Carolina, United States, 253–264. <https://doi.org/10.1145/192844.193021>
- [106] Susan Leigh Star and Karen Ruhleder. 1996. Steps Toward an Ecology of Infrastructure: Design and Access for Large Information Spaces. *Information Systems Research* 7, 1 (March 1996), 111–134. <https://doi.org/10.1287/isre.7.1.111>
- [107] John Stark. 2016. The PDM System. In *Product Lifecycle Management (Volume 2)*. Springer International Publishing, Cham, 233–252. https://doi.org/10.1007/978-3-319-24436-5_11 Series Title: Decision Engineering.
- [108] Rainer Stark. 2022. Major Technology 1: Computer Aided Design—CAD. In *Virtual Product Creation in Industry*. Springer Berlin Heidelberg, Berlin, Heidelberg, 113–138. https://doi.org/10.1007/978-3-662-64301-3_7
- [109] Rainer Stark. 2022. Major Technology 5: Product Data Management and Bill of Materials—PDM/BOM. In *Virtual Product Creation in Industry*. Springer Berlin Heidelberg, Berlin, Heidelberg, 223–272. https://doi.org/10.1007/978-3-662-64301-3_11
- [110] Sarah Sterman, Molly Jane Nicholas, and Eric Paulos. 2022. Towards Creative Version Control. *Proceedings of the ACM on Human-Computer Interaction* 6, CSCW2 (Nov. 2022), 1–25. <https://doi.org/10.1145/3555756>
- [111] Anselm Strauss. 1988. The Articulation of Project Work: An Organizational Process. *The Sociological Quarterly* 29, 2 (June 1988), 163–178. <https://doi.org/10.1111/j.1533-8525.1988.tb01249.x>
- [112] Eswaran Subrahmanian, Ira Monarch, Suresh Konda, Helen Granger, Russ Miliken, Arthur Westerberg, and Then-dim Group. 2003. Boundary Objects and Prototypes at the Interfaces of Engineering Design. *Computer Supported Cooperative Work (CSCW)* 12, 2 (June 2003), 185–203. <https://doi.org/10.1023/a:1023976111188> Publisher: Springer Science and Business Media LLC.
- [113] C Sun, D Guo, H Gao, L Zou, and H Wang. 2011. A method of version merging for computer-aided design files based on feature extraction. *Proceedings of the Institution of Mechanical Engineers, Part C: Journal of Mechanical Engineering Science* 225, 2 (Feb. 2011), 463–471. <https://doi.org/10.1243/09544062JMES2159>
- [114] Dassault Systemes. 2024. Searching with the SOLIDWORKS PDM Search Tool. https://help.solidworks.com/2025/english/EnterprisePDM/fileexplorer/_Searching_in_Vaults_search_tool.htm?verRedirect=1
- [115] Hongtao Tang, Shunsheng Guo, Lang Huang, Li Li, and Yibing Li. 2015. Research and development on key models and technology of PDM system. *The International Journal of Advanced Manufacturing Technology* 78, 9-12 (June 2015), 1865–1878. <https://doi.org/10.1007/s00170-014-6765-8>
- [116] Thangs 3D. 2022. Thangs Workspace. <https://blog.thangs.com/presenting-thangs-workspace-3c79416ba2d8>
- [117] P. Van Den Hamer and K. Lepoeter. 1996. Managing design data: the five dimensions of CAD frameworks, configuration management, and product data management. *Proc. IEEE* 84, 1 (Jan. 1996), 42–56. <https://doi.org/10.1109/5.476025>
- [118] Christian Voigt. 2018. Not Every Remix is an Innovation: A Network Perspective on the 3D-Printing Community. In *Proceedings of the 10th ACM Conference on Web Science*. ACM, Amsterdam Netherlands, 153–161. <https://doi.org/10.1145/3201064.3201070>
- [119] Ademir-Paolo Vrolijk, Yuanzhe Deng, and Alison Olechowski. 2023. CONNECTING DESIGN ITERATIONS TO PERFORMANCE IN ENGINEERING DESIGN. *Proceedings of the Design Society* 3 (July 2023), 1067–1076. <https://doi.org/10.1017/pds.2023.107>
- [120] R. Watkins and M. Neal. 1994. Why and how of requirements tracing. *IEEE Software* 11, 4 (July 1994), 104–106. <https://doi.org/10.1109/52.300100>
- [121] I.C. Wright. 1997. A review of research into engineering change management: implications for product design. *Design Studies* 18, 1 (Jan. 1997), 33–42. [https://doi.org/10.1016/S0142-694X\(96\)00029-4](https://doi.org/10.1016/S0142-694X(96)00029-4)
- [122] David C. Wynn and Claudia M. Eckert. 2017. Perspectives on iteration in design and development. *Research in Engineering Design* 28, 2 (April 2017), 153–184. <https://doi.org/10.1007/s00163-016-0226-3> Publisher: Springer London.
- [123] YoungSeok Yoon, Brad A. Myers, and Sebon Koo. 2013. Visualization of fine-grained code change history. In *2013 IEEE Symposium on Visual Languages and Human Centric Computing*. IEEE, San Jose, CA, USA, 119–126. <https://doi.org/10.1109/VLHCC.2013.6645254>
- [124] Jingyi Zhang, Jiaxing Huang, Sheng Jin, and Shijian Lu. 2024. Vision-Language Models for Vision Tasks: A Survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 46, 8 (Aug. 2024), 5625–5644. <https://doi.org/10.1109/TPAMI.2024.3369699>
- [125] Jian Zhang, Xu Wang, Hongyu Zhang, Hailong Sun, and Xudong Liu. 2020. Retrieval-based neural source code summarization. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*. ACM, Seoul South Korea, 1385–1397. <https://doi.org/10.1145/3377811.3380383>
- [126] Lei Zhang, Ashutosh Agrawal, Steve Oney, and Anhong Guo. 2023. VRGit: A Version Control System for Collaborative Content Creation in Virtual Reality. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*. ACM, Hamburg Germany, 1–14. <https://doi.org/10.1145/3544548.3581136>
- [127] Jinxuan (Janice) Zhou, Vrushank Phadnis, and Alison Olechowski. 2021. Analysis of Designer Emotions in Collaborative and Traditional Computer-Aided Design. *Journal of Mechanical Design* 143, 2 (Feb. 2021), 021401. <https://doi.org/10.1115/1.4047685>